实验要回答什么：两次实现变化，分别验证

问题 A：保存路径

同步保存 → 异步保存

完整文件是否更早就绪？
主流程负担转移到哪里？
质量检查是否仍通过？

主要证据：v1 四配置对照

问题 B：规划后端

v1 auto → v2 auto

接口怎样迁移？
能否完整生成并校验？
是否建立性能优势？

主要证据：v2 新重跑

讲解顺序

先交代实验与计时口径
再说明实现机制
然后读效率与内部耗时
最后讨论质量和局限

主线 45 页，附录 6 页

先分清保存优化和规划迁移两个问题

怎么做的	实验沿两条路线展开：先保留 cuRobo v1 规划与仿真任务，比较同步保存和异步保存；再在异步保存基础上接入 cuRobo v2。每轮记录整批文件最终就绪时间，并检查保存数据的结构和任务末态。
这样做的意义	生成数据既要快，也要可用。把保存改造与规划迁移分别比较，才能判断收益主要来自减少生成阻塞，还是来自规划方法变化；同时避免用保存条数掩盖任务质量变化。
当前结论	已有结果支持 v1 异步保存改善本任务的生产效率。v2 已完成接口接入与整轮运行，但现有结果没有建立其效率和质量优势。后文先解释改动机制，再用运行记录检验。
读图与术语	episode 指一次任务尝试；writer 指压缩、写文件的 CPU 子进程；规划后端负责从目标求出机器人运动轨迹。

实验矩阵：哪些条件相同，哪些条件不同

记录组	条件 / 范围	改变因素	样本与验收
v1 同步	10/14 · 8 环境 · seed 101	同步保存；0 writer	17 完成 / 16 保存 / 16 末态通过
v1 cap4 / cap8 / auto	10/14 · 8 环境 · seed 101	异步策略不同；实际均 4 writer	各 17 完成 / 16 保存 / 16 通过
v2 auto 重跑	10/15 · 8 环境 · seed 101	规划后端变化；实际 4 writer	16 完成 / 16 保存 / 15 通过
早期生产 / 离线保存	24 条生产 / 4 条离线	同步、单 writer、双 writer	不同批次；只作补充证据
离线压缩对照	1342 帧 / 相机；固定 chunk	gzip / none / lzf	仅比较保存耗时与体积

哪些实验可以直接比较

怎么做的

历史主实验使用 8 个仿真环境、seed 101，每组目标保存 16 条，比较 v1 同步、cap4、cap8 和 auto。v2 auto 在次日运行；新增内部计时使用独立的 4 环境、8 条目标配对，放在第 29–36 页。

这样做的意义

环境数、保存目标、日期和计时插桩都会影响运行时间。先列出共同条件和改变因素，才能确定每个百分比对应哪一组比较，避免把不同规模的实验拼成一个平均值。

当前结论

历史 v1 四组能用于观察同一任务下保存方式的变化；v2 跨日期比较只能描述这次运行。各历史配置只有一轮，细小名次差异没有重复试验支持；4 环境配对提供独立的内部耗时证据。

读图与术语

cap 是允许启动的 writer 数量上限，不等于实际进程数。本轮 cap4 与 cap8 都只启动了 4 个 writer。

指标口径：同一条数据，可以有不同的完成时刻

整批墙钟时间

T = 首条开始 → output_ready
包含生成与最终收尾
不含首条开始前全部启动

完整启动退出用 launcher 时间
不要混用两个计时边界

吞吐与质量分母

记录成功吞吐 = 16 × 60 / T
末态合格吞吐 = 通过数 × 60 / T

v1：末态通过 16 条
v2：末态通过 15 条
不能把保存成功直接当质量合格

内部阶段计时

v1：17 个完成 episode 的均值
v2：16 个完成 episode 的均值
后台：每组 16 次保存事件

并发 episode 的耗时重叠
阶段均值之和 ≠ 整批墙钟

时间从哪里开始算，吞吐如何计算

怎么做的	最终就绪时间 T 从第一条 episode 开始计到 output_ready：后台任务已排空、分片已合并、最终输出可用。记录成功吞吐等于保存条数×60/T；末态合格吞吐则将分子换成严格检查通过的条数。
这样做的意义	异步提交可以很快返回，但最终文件可能仍在写入。把终点固定在输出可用时，才能公平比较同步与异步；再用质量通过数作分子，才能回答每分钟真正获得多少合格数据。
当前结论	整批墙钟时间适合衡量生产效率；每个 episode 的阶段均值适合解释等待发生在哪里。多个环境的计时区间会重叠，不能将这些均值或后台写入耗时相加来还原整批时间。
读图与术语	墙钟时间就是现实经过的时间。launcher_seconds 另包含首条任务开始前的启动成本，与本图 T 的起点不同。

先区分保存路径改造与规划后端迁移

原始版：v1 同步基线	改进版：v1 异步保存	cuRobo v2 适配版
本报告的对照实现	主要改变保存侧	主要改变规划侧
规划：MotionGen 环境：8 环境生成配置 保存：主流程同步导出	规划：保留 v1 后端 环境：保留 8 环境 保存：独立 CPU writer	规划：MotionPlanner 环境：保留 8 环境 保存：沿用异步保存池
压缩与写入完成后 生成控制流才能继续	共享内存移交数据 分片保存 → 排空 → 合并	适配模型与目标表示 插值轨迹 → 关节重排 → FK
已使用 GPU 与多环境 不是“整个系统完全串行”	生成—保存流水线重叠 另增加保存侧任务并行	规划后端迁移与接口适配 不是新增 GPU 生产进程

三种实现分别改了什么

怎么做的	原始 v1 使用 MotionGen 规划，主流程同步导出；改进 v1 保留规划和仿真任务，把保存交给 CPU writer；v2 适配版沿用异步保存，改变规划配置、目标类型及结果转换。图中三列对应本项目测试实现。
这样做的意义	代码版本名称容易让人误以为每次升级都改变整个系统。明确保存侧和规划侧的边界，可以解释为什么保存优化无需更换规划器，也能改善整批效率。
当前结论	原始 v1 已经使用 GPU 和多环境仿真。改进 v1 的主要变化是生成与保存能够重叠；v2 的主要变化是规划接口迁移。两项改动应各自对照，不能把保存收益全部归于新版规划库。
读图与术语	本页及历史主线使用 8 环境配置；后文新增的 4 环境内部计时实验单独标注。

沿调用链说明具体改动

原始版：同步保存



同步导出占用生成控制流。

改进版：异步保存



Bridge.export → ExportPool.submit；提交后生成可继续，超预算仍会等待。

v2 版：规划适配 + 异步保存



规划路径在前端变化；episode 保存仍走共享内存与 writer 池。

沿调用链找到阻塞发生的位置

怎么做的	同步版沿录制器调用进入压缩和 HDF5 写入，返回后继续生成。异步版先由 Bridge.export 打包，再由 ExportPool.submit 提交，writer 后台写分片，主流程通过回执、排空和合并完成收尾；v2 在前端增加 plan_pose 及轨迹转换。
这样做的意义	跟随数据和返回时刻，比只看模块名称更容易理解加速机制：保存从生成控制流中移开后，主流程可以继续准备动作；内存预算不足时仍需等待，保存成本并未消失。
当前结论	改动确实将部分保存工作转交后台，并保留最终输出完成条件。图中箭头表达调用依赖，不表示各框按比例耗时，也不能用来证明 GPU 上的实际重叠。
读图与术语	HDF5 是本实验的数据文件格式；分片是多个 writer 各自写出的中间文件，最终需要合并。

代码差别在导出路径和扩容决策，共用 cuRobo v1 生成任务

同步 · workers=0	固定上限 · 4 / 8	auto
保留原 recorder 导出	ExportPool._scale_workers	ResourceGovernor.decide
episode 完成 ↓ 主进程压缩 + 写 HDF5 ↓ 继续生成	pending 有积压 ↓ desired = min(pending, cap) ↓ 按需启动 CPU writer	pending 有积压 ↓ CPU / 内存 / I/O 等准入 ↓ 通过：增加 1 个 writer
主流程承担保存等待	本轮两种上限均启动 4 个	至少间隔 1 秒；不主动缩容

同步、固定上限与 auto 的差别

怎么做的	workers=0 使用原同步导出。固定上限模式根据待保存任务数按需启动 writer，数量不超过 cap；auto 模式还检查 CPU、内存、I/O 等资源准入，通过后逐步增加 writer。本轮 auto 不主动缩容。
这样做的意义	writer 太少可能积压，太多也可能争抢 CPU、内存和磁盘。固定上限提供简单约束，auto 尝试在有积压且资源允许时扩容，两者影响的是保存侧容量。
当前结论	本轮三种异步配置均实际启动 4 个 writer。因此可以观察异步机制的整体收益，却不能把 cap8 与 cap4 的差异解释成 8 个进程优于 4 个进程，也不能分别量化各项内存改动的贡献。
读图与术语	pending 指尚未完成的保存任务；auto 的资源准入不等于自动增加仿真环境或 GPU。

模块拆分：原 handler 的保存职责转入桥接层与进程池

原始：生成进程内

RecorderManager
录制 / export_episodes
↓
EpisodeData.pre_export
列表统一 torch.stack
↓
HDF5DatasetFileHandler
write_episode + flush

改进：生成主进程

RecorderManager（保留）
↓ 实例方法转接
Bridge + MetadataHandler
导出路由 / 元数据
↓
ExportPool
预算 / 任务 / 内存 / 回执
最终排空与合并

改进：CPU 子进程

spawn → writer
↓ 消费 jobs 描述信息
write_job
映射 CPU 共享内存
gzip + 独立 HDF5 分片
↓
results 返回 saved / failed
不初始化仿真与 CUDA

生成、移交、写入和合并如何分工

怎么做的	原录制器负责收集 EpisodeData，并调用 HDF5 handler 导出。改进版在生成侧接入桥接层，保存池负责预算、排队与进程管理，writer 接收共享内存描述并写独立分片，主流程最后核对回执和合并文件。
这样做的意义	把职责拆开后，生成不必亲自完成每一次压缩写入；同时，每块数据由谁持有、失败由谁处理也有明确边界，便于定位积压和内存增长。
当前结论	实现已经形成生成主进程与 CPU 保存子进程的分工。这个结构支持流水线重叠，但图本身不提供各模块独立的提速比例；函数映射和冻结源码摘录见 code-structure.html。
读图与术语	共享内存用于让不同进程访问同一块已打包数据；描述信息包含形状、类型及位置，并非重新生成一次轨迹。

四个接入时刻：安装异步保存，不永久改写上游源码

创建环境前后

generate.build_source
按固定锚点改入口文本

创建前 configure(env_cfg)
替换类方法 add_to_episodes
设置 MetadataHandler 类型

创建后 attach(recorder, settings)
MethodType 绑定实例 export

生成运行中

compat.install_generation_compat
在 env_loop 中插入 poll

外层循环处理回执
等待动作时也处理回执

Bridge.export → pool.submit
主流程仍检查预算和打包
提交返回不等于保存完成

结束或异常时

达到生成目标 → finish
检查成功提交数
排空 → join → merge
关闭环境 → output_ready

异常 / finally / atexit
调用 abort 清理自有资源
锚点变化需重新核对适配

改进如何接入原来的运行流程

怎么做的	启动器 build_source 在内存中构造接入后的运行代码：环境创建前 configure 保存配置，创建后 attach 录制桥接，运行中 poll 处理 writer 回执，结束时 finish 排空并合并。类方法包装和环境实例绑定分别完成接入。
这样做的意义	接入点必须覆盖完整生命周期。只有提交而没有持续回收或最终排空，会造成内存滞留或提前宣布完成；保留独立启动入口也方便比较原始与改进实现。
当前结论	现有实现通过启动注入和运行时绑定接入保存池，上游源码文件没有因此被永久改写。复现实验应使用冻结启动器与相同开关，而非只复制某一个 writer 函数。
读图与术语	hook 指在既有调用点接入包装逻辑；poll 是主流程主动检查后台结果，不是再次保存同一条数据。

数据所有权：环境切片、源帧与共享内存分别管理

原始数据路径

录制：整批 value.clone()
每环境保留切片 clone=False
切片可继续持有整批存储

导出：pre_export
各字段列表 torch.stack
.cpu().numpy() → HDF5
统一堆叠仍在生成进程

改进 CPU 帧路径

录制：每环境独立 clone
导出：layout 计算偏移与形状
分配有界共享内存

逐帧 np.copyto
已复制源帧置 None / 清空
尝试归还分配器空闲页
不调用原统一 pre_export

必须区分的边界

GPU 状态列表仍先 stack
再转 CPU、复制到共享内存

源帧复制后可释放
共享内存等 saved 后释放

不是零拷贝 / CUDA IPC
没有独立峰值内存消融

为什么要调整录制缓冲的所有权

怎么做的	改进版让 CPU 图像按环境持有独立数据，避免某个环境的切片继续引用整个批次。打包到共享内存时逐帧移交并释放可回收的源帧；小型 GPU 状态仍先堆叠，再复制到 CPU。
这样做的意义	多环境、长轨迹和多相机会同时保留大量帧。若小切片仍持有大批次底层存储，一个 episode 保存完也未必释放其对应内存，容易抬高峰值并触发预算等待。
当前结论	实现减少了不必要的整批引用保留，但仍存在 CPU 复制和共享内存打包。配对实验观测到内存峰值降低；因为同时改变了保存路径，尚不能将全部下降单独归因于这一项。
读图与术语	所有权指哪一方负责保持数据有效及释放它；切片和独立副本的内存生命周期不同。

完成语义变化：提交、分片保存和最终就绪分成三步

提交与保存

submit → entries=submitted
任务加入 pending 与 jobs

writer：started → write_job
写临时组 → 移动 → flush
返回 saved

poll 才释放内存并更新计数

最终文件

finish 等待 pending 清空
发送退出哨兵并 join

_merge 排序复制分片
检查属性 / shape / dtype
.partial → os.link 最终路径

env.close 后记录 output_ready

失败与限制

failed / 进程退出 / 超时
由父进程 poll 报错
abort 终止 writer 并回收内存

合并检查不等于逐值校验
没有多输出事务回滚保证
没有 fsync 持久化承诺
末态质量仍需独立验收

提交成功之后，何时才算保存完成

怎么做的	submit 接受任务后计为 submitted；writer 写完分片并回执后计为 saved；finish 等待所有任务结束，合并、关闭最终输出后才发出 output_ready。失败、超时或 writer 异常退出由主流程处理，并通过 abort 清理资源。
这样做的意义	异步系统同时存在多个完成时刻。如果把提交数当作最终保存数，表面吞吐会很好看，但可能还有未写完数据；异常传播和清理则防止坏任务被悄悄计为成功。
当前结论	本实验以最终输出就绪比较效率，并结合文件回读和质量检查确认结果。后台写入并没有取消数据完整性责任，后续优化也必须保留这条完成链。
读图与术语	output_ready 是整批最终可用标记；saved 是单个保存任务的回执，两者对应不同范围。

流程分段：并行化发生在哪里，哪里仍需等待

阶段	执行方式	同步点与边界
任务调度	asyncio 协程并发	共享主线程；等待未完成时挂起
规划计算	GPU 算子内部数据并行	主机逐环境调用；跨请求重叠未测
仿真步进	动作汇合后批量 env.step	等待整批动作；不是每环境一个进程
录制 / 内存移交	主流程 clone、pack、submit	CPU 帧所有权调整；仍有打包与预算等待
压缩 / 写入	spawn CPU 进程处理不同 episode	独立 HDF5 分片；共享 CPU 与磁盘资源
回执 / 收尾	主流程 poll、finish、merge	回执后回收内存；排空后合并最终输出

系统中有四种不同的并发

怎么做的

逐段检查执行单位：asyncio 调度多个环境任务；规划和物理算子在 GPU 内部进行数据并行；动作汇合后统一调用批量 env.step；CPU writer 使用独立进程压缩写文件。录制、打包与回执仍由主流程参与。

这样做的意义

这些并发解决不同问题。协程允许任务在等待时交出控制权，GPU 并行处理数值计算，多进程保存利用 CPU 资源；增加其中一种数量，不会自动增加另一种的并行度。

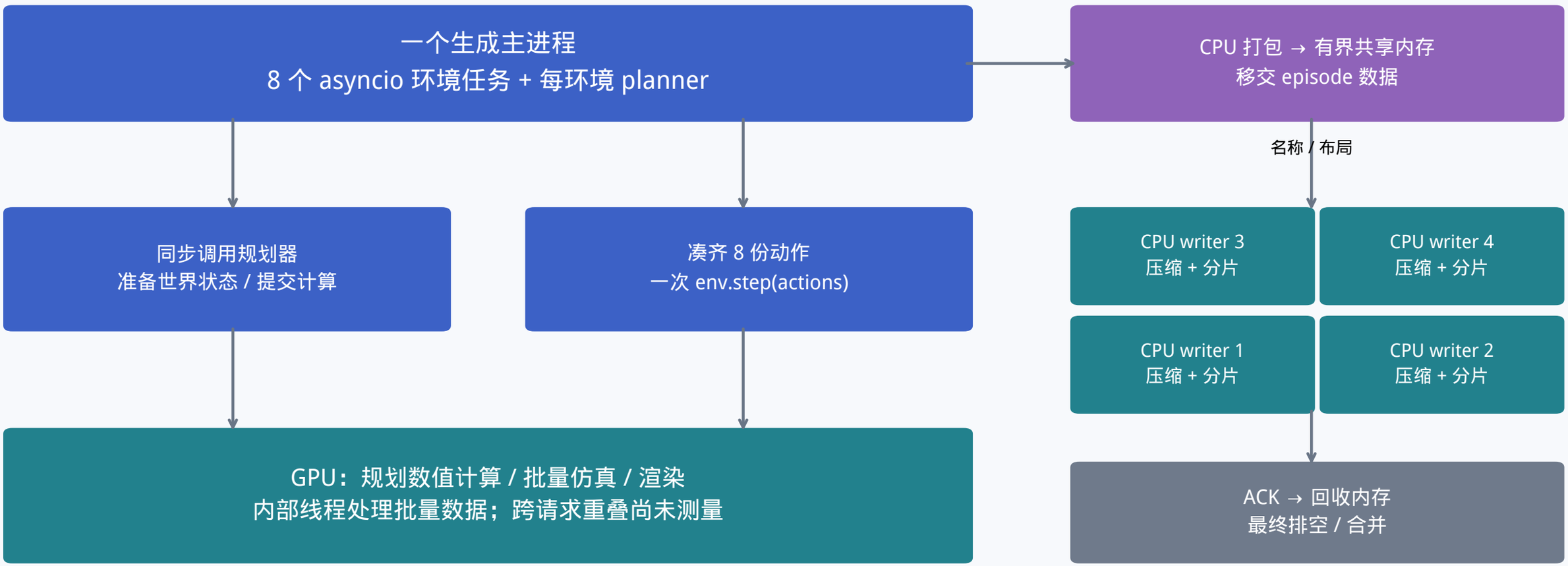
当前结论

明确新增的独立执行者是 CPU writer。规划请求之间、物理与图形引擎之间是否实际重叠，仍需设备时间线确认；源码中的异步接口名称不足以证明重叠。

读图与术语

协程共享主线程，不能让主线程中的同步压缩同时执行；GPU 算子内部并行也不等于多个规划请求同时运行。

执行层次：协程并发、GPU 数据并行、CPU 多进程



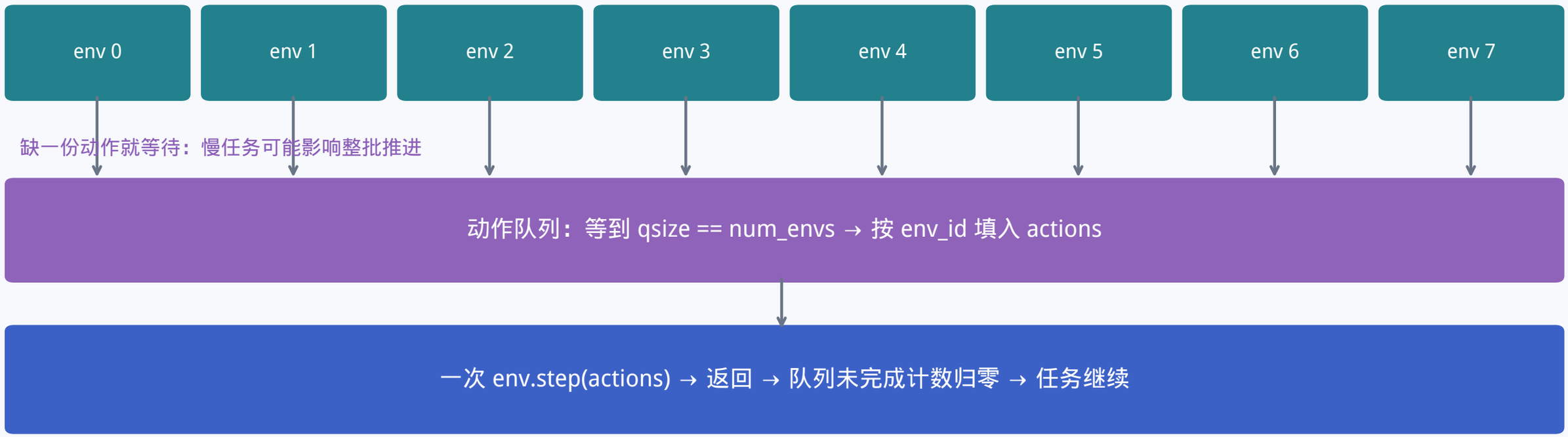
架构示意：8 个环境任务共享生成主进程；本轮异步组实际均为 4 个 CPU writer。

一个生成进程与多个保存进程

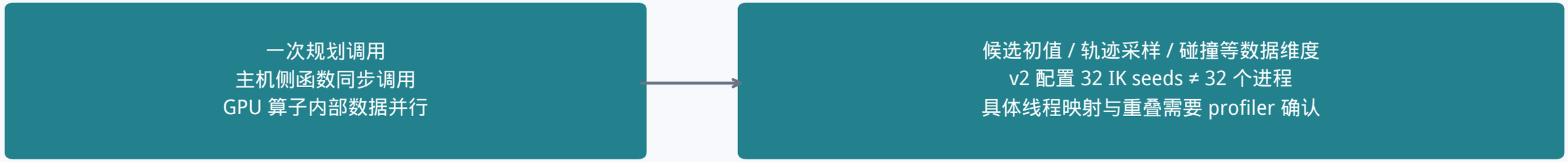
怎么做的	历史实验在同一个生成主进程中管理 8 个仿真环境，并调用 GPU 规划、物理和渲染；三种异步配置实际启动 4 个 CPU writer。图中的共享内存连接生成侧与保存侧，结果回执沿反方向返回。
这样做的意义	理解进程边界，可以解释异步保存为何释放生成控制流，也能判断瓶颈是否可能来自 CPU 或磁盘争用，而不是一律归因于 GPU 算力。
当前结论	writer 不启动新仿真环境，也不承担 CUDA 规划。提高保存进程上限是保存侧调度调整；本实验没有验证多个 GPU 生成进程的部署收益。
读图与术语	环境是不同任务状态的仿真实例；进程是操作系统执行单元。8 个环境可以由一个主进程统一推进。

环境在动作屏障汇合；GPU 算子内部执行数据并行

主进程：环境提交动作后等待 queue.join()；等待未完成时挂起协程



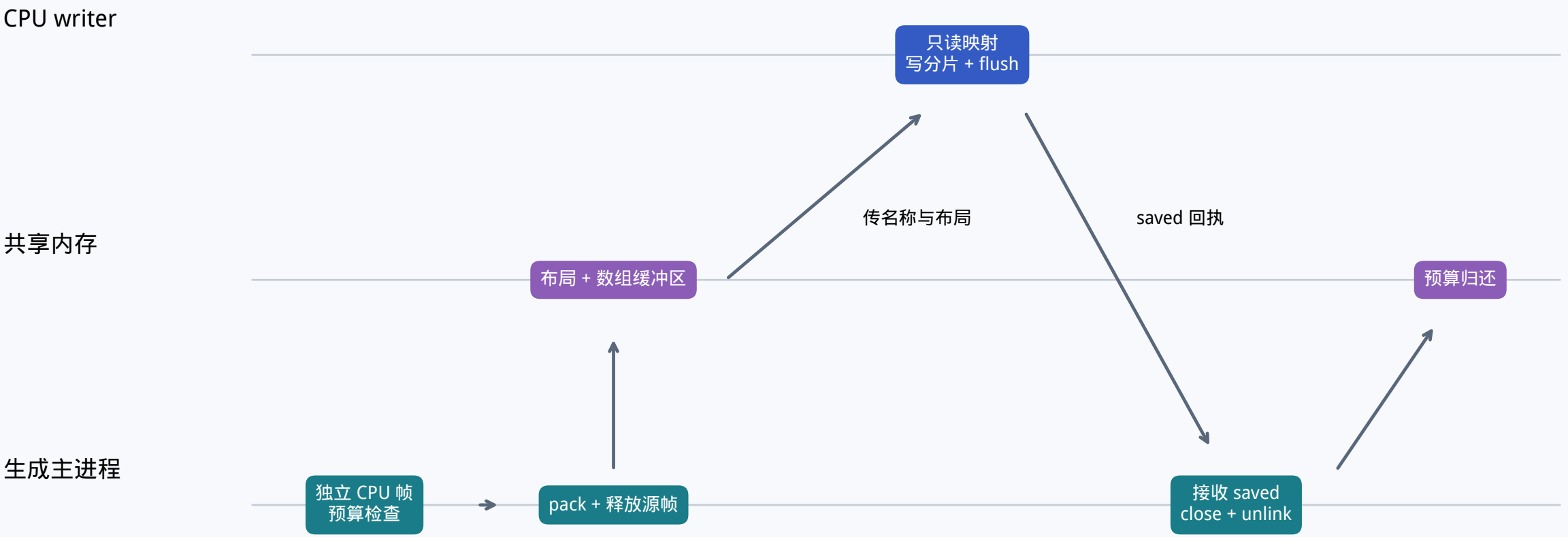
下方是规划计算原理示意，与上方动作屏障不是同一条执行时间线。



为什么一个环境的等待会影响整批

怎么做的	各环境任务将动作放入队列；动作数量达到环境数后，主循环执行一次批量 env.step，再通过 task_done 更新队列完成记账。execute 中的 queue.join 等待这些已提交动作被处理。
这样做的意义	统一步进便于批量仿真，却让动作准备和主循环调度成为共同依赖。某个环境尚未提交动作，或主线程正在同步保存，都可能推迟这一批的推进。
当前结论	代码支持主机侧批次等待这一机制解释。queue.join 的等待区间包含主循环完成有用工作的时间，不能直接当作 GPU 空闲；其下降也不能单凭本页分解为某一种设备等待减少。
读图与术语	barrier 指凑齐一批条件后才能继续；队列记账是主机逻辑，不是测量 CUDA 完成时间的设备事件。

数据所有权与回收：必须等 writer 确认后才能释放



示意图，无时间比例；队列预算不足会等待回执，writer 写入可与下一条生成重叠。

一条数据从录制到释放的生命周期

怎么做的	录制产生源帧；主流程将其打包到共享内存后，可释放不再使用的原 CPU 帧。writer 从共享内存写出分片并发送 saved 回执；主流程收到回执后释放共享内存，并归还在途字节预算。
这样做的意义	生产速度长期快于保存速度时，未完成数据会越积越多。有界预算将积压限制在可承受内存内；正确的释放时刻保证 writer 读取期间数据仍然有效。
当前结论	源帧释放与共享内存释放是两个不同事件。现有流程提供有界积压和完成后回收，代价是预算不足时生成侧仍需等待，不能追求无限排队来换取短期生成速度。
读图与术语	在途数据指已经提交但尚未回收的保存数据；预算限制的是资源占用，不是最终数据集大小。

异步保存带来了多少最终产出收益

怎么做的

对历史 v1 四组使用相同的 16 条保存目标，读取从首条开始到最终输出就绪的时间，再换算为每分钟保存条数。同步组为基线，cap4、cap8 和 auto 均走异步保存。

这样做的意义

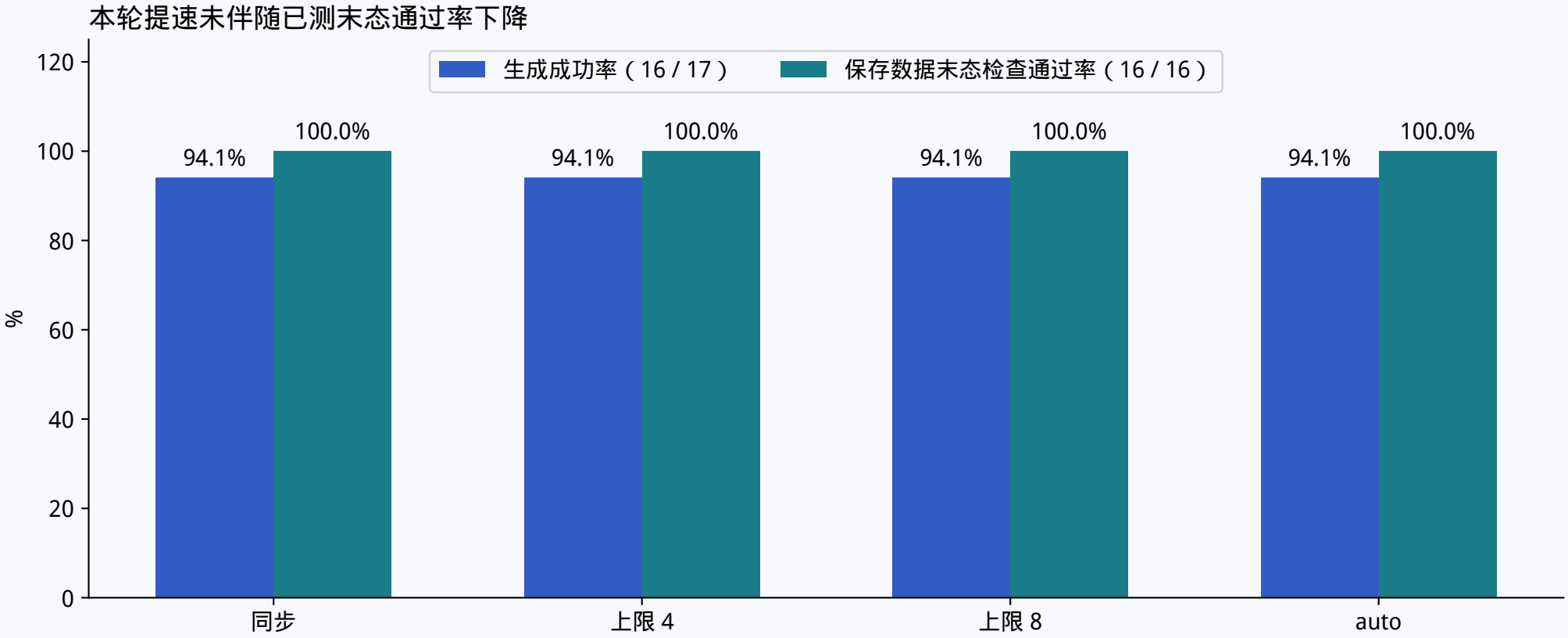
这是用户实际等待结果的指标，包含后台排空和合并，能够检验生成阶段省下的时间是否在收尾阶段又被消耗掉。

当前结论

这轮 8 环境实验中，异步组最终吞吐相对同步提高约 55%–66%，auto 提高约 60.2%。结果支持保存改造有效；各配置只有一轮且实际均为 4 writer，不能据此确定长期最优上限。

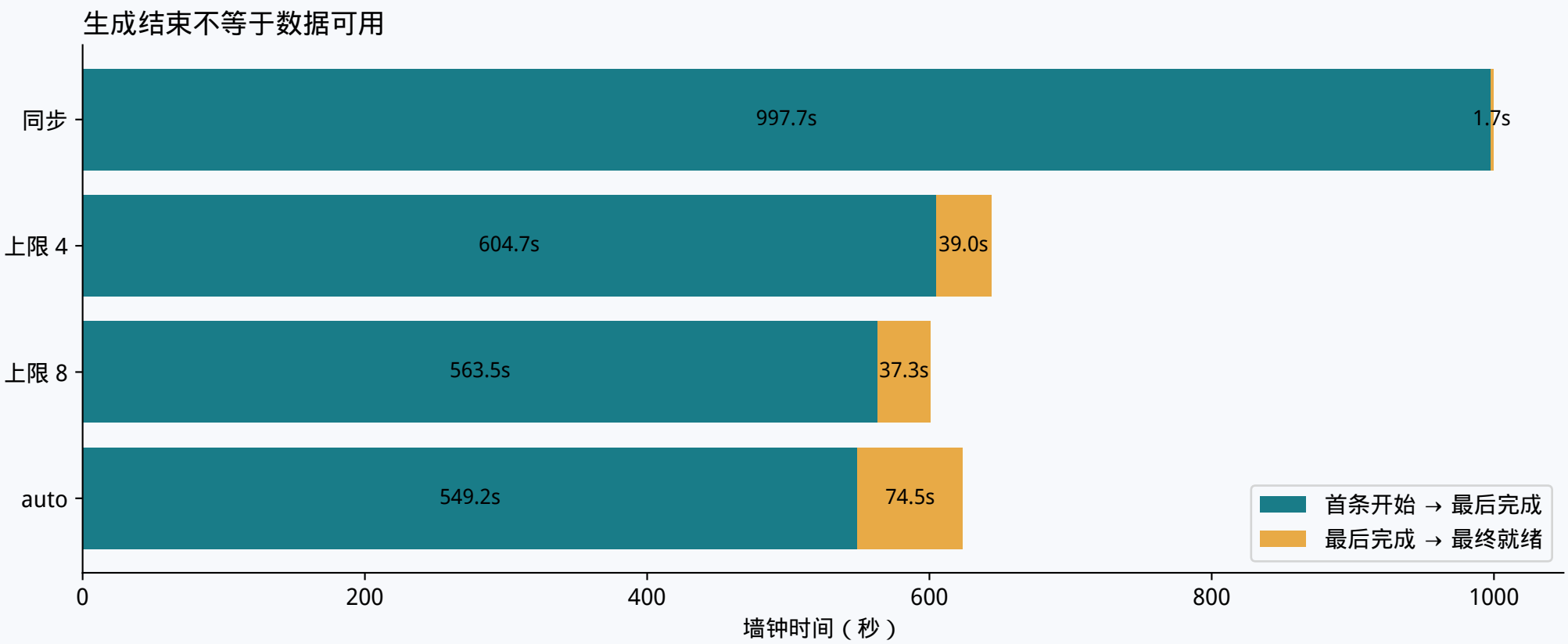
读图与术语

耗时减少比例与吞吐提高比例不同：相同产出下，吞吐与时间成反比，不要把两种百分比混用。



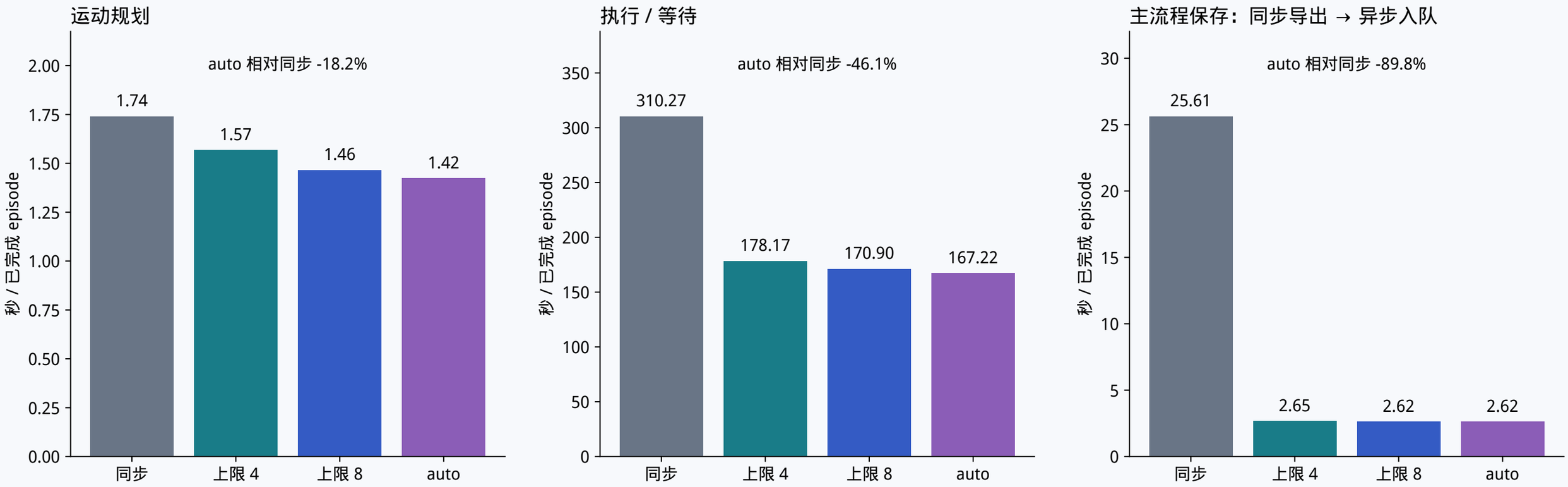
变快之后，数据仍然合格吗

怎么做的	分别统计任务尝试结果、实际保存条数、HDF5 结构回读和严格末态检查。历史 v1 各组 17 个已完成 episode 中有 1 次生成失败，最终保存 16 条；16 条保存数据均通过末态检查。
这样做的意义	只有速度而没有质量约束，可能把提前结束或错误状态计为优化收益。将生成成功和保存后质量分开，才能判断获得的是否是可用训练数据。
当前结论	本轮 v1 各组保存数据均为 16/16 合格，支持观察到的效率收益没有伴随这些检查项退化。样本较小，不能推断所有场景的长期成功率；取消任务也需独立统计。
读图与术语	生成失败计入已完成尝试，但通常不进入成功保存集合；严格末态检查验证最终物理状态，文件结构检查验证数据格式。



为什么生成完成后还要继续等待

怎么做的	将整批时间分为最后一个已完成 episode 结束之前的生成窗口，以及其后到 output_ready 的尾段。尾段由剩余后台保存、分片合并和文件关闭等工作构成。
这样做的意义	异步保存将工作挪到后台，可以缩短生成阻塞，却可能留下更多收尾工作。观察尾段才能判断端到端收益，避免仅用“最后一条轨迹生成完”作为终点。
当前结论	历史 auto 本轮尾段约 74.48 秒，说明收尾值得继续拆分。它仍在最终吞吐上优于同步；一次较长尾段并不能证明 auto 策略必然导致较差收尾。
读图与术语	尾段按整批独占的墙钟区间计算；后台写入在生成窗口内已经进行的部分，不应再次全部相加。



各组 17 个完成样本（含 1 个失败）；右图两端工作不同，反映主流程负担而非后台写入加速。

episode 阶段均值为什么会下降

怎么做的	从历史已完成 episode 日志中分别取规划、execute、导出或入队等阶段，计算每组均值；v1 各组统计 17 个完成尝试，包括一次失败。异步入队与同步完整导出的工作范围在图中分别标注。
这样做的意义	整批收益需要用阶段变化解释。保存工作占用主控制流时，其他环境可能仍处于 execute 等待中；移走保存工作既能改变导出阶段，也可能缩短观察到的执行等待。
当前结论	同步导出均值约 25.6 秒，auto 入队约 2.6 秒，但后者还不包含后台写入完成。execute 均值的下降支持主循环受阻减少的解释；内部具体归因由第 29–32 页的插桩配对补充。
读图与术语	阶段均值用于比较同类区间；多个环境会重叠，不能将这些条形累计成整批最终时间。

执行 / 等待 = MultiWaypoint.execute 整段调用的墙钟时间

计时包含的顺序

time.monotonic() 开始
↓
目标 / 噪声 → 动作转换
↓
动作 put → await queue.join
↓
成功条件检查 → bool
↓
正常返回后累计耗时

queue.join 期间可能发生

等待其余环境提交动作
整批 env.step(actions)
仿真 / 观测 / 录制等路径
队列完成后的协程恢复延迟

其他环境同步规划或导出
也可能占用共同事件循环线程

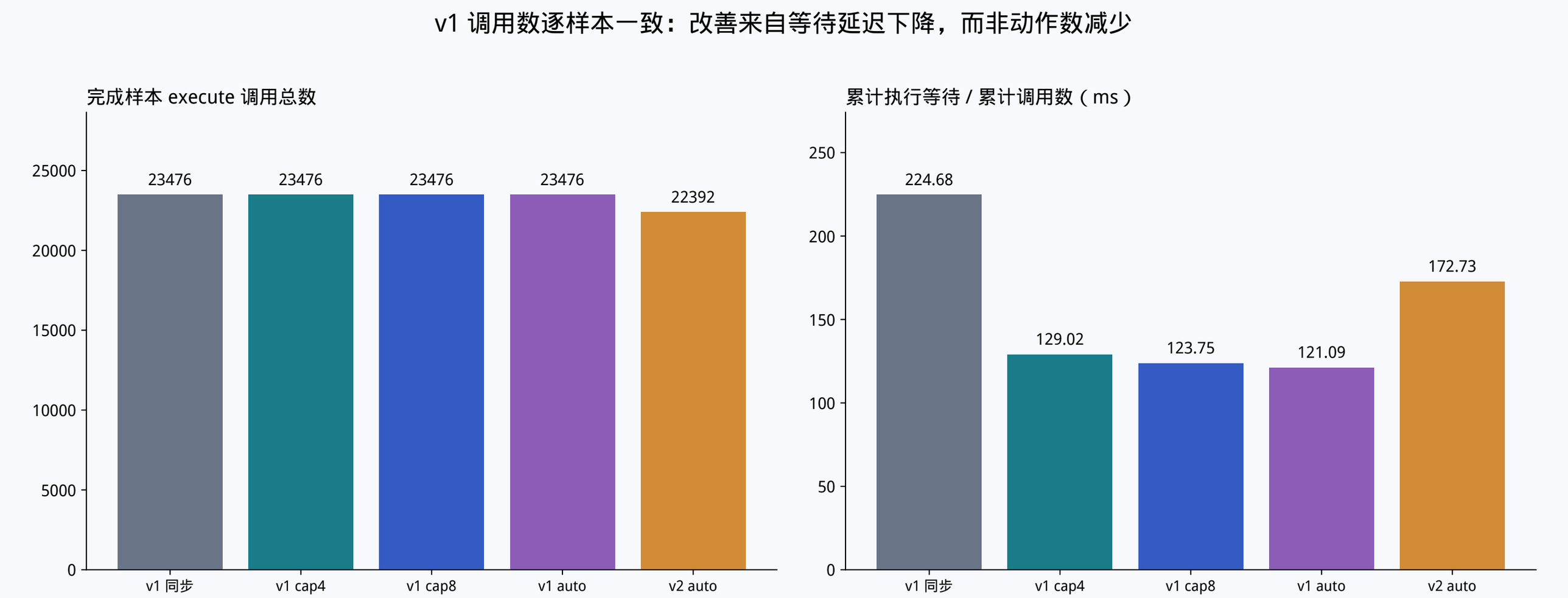
旧日志无法直接拆出的量

纯 physics kernel 时间
纯渲染与相机处理时间
动作屏障占比
主机与设备同步占比

同一等待可被多个环境累计
不能按 GPU 忙碌时间解释

execute 计时到底覆盖什么

怎么做的	历史计时器包住整个 await execute 调用：准备或提交动作后，等待动作队列的完成条件。此时主循环可能在推进其他环境、调用物理、更新观测，甚至执行同步导出。
这样做的意义	先理解计时边界，才能回答“执行等待的大部分在做什么”。函数名中的 execute 不是单一物理内核，await 的经过时间也不是该协程独占 CPU 或 GPU 的时间。
当前结论	历史 execute 指标适合观察调用链延迟，却不能直接分解成 GPU 物理、渲染与空闲比例。新增插桩将主线程区间按实际操作归类，补上内部证据；queue.join 与内部工作不能重复相加。
读图与术语	await 表示当前协程可以挂起并交还调度权；等待期间其他任务可以继续工作。



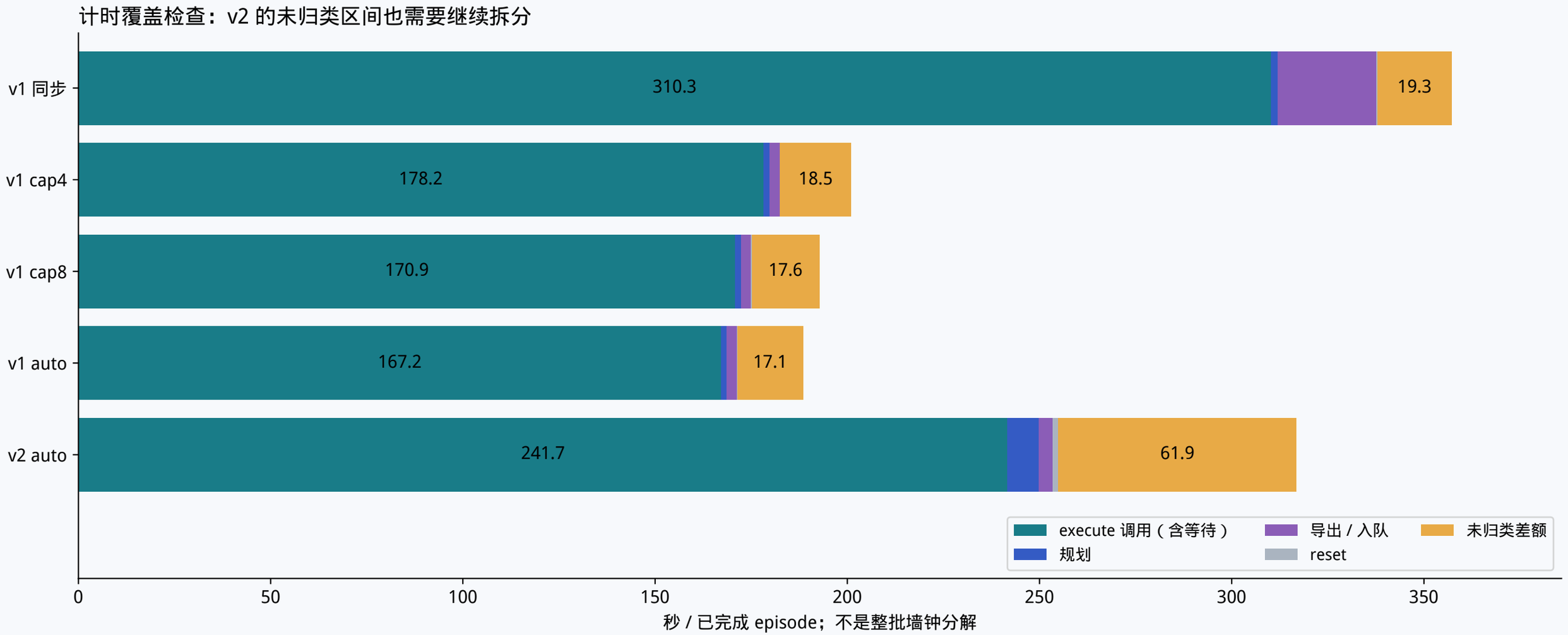
是否通过少执行动作获得了加速

怎么做的	比较各组 execute 调用次数与累计耗时，再计算每次调用的平均经过时间。历史 v1 四组均有 23,476 次调用，并按各自已完成 episode 数量提供归一化对照。
这样做的意义	单看执行总时长下降，可能误以为机器人少走了步骤。调用次数提供工作量参照，有助于区分相同调用数量下延迟降低与执行数量减少。
当前结论	历史 v1 四组调用数相同，观察到的时间差主要体现为单次调用经过时间变化。相同次数仍不能证明轨迹、图像和物理末态完全一致；新增配对进一步验证了完整动作数组与末态。
读图与术语	v1 的 17 个完成尝试与 v2 的 16 个完成尝试统计范围不同；跨版本比较应同时读取分母和质量结果。

区分典型速度与长时间停顿：窗口 P95 不是单次调用 P95

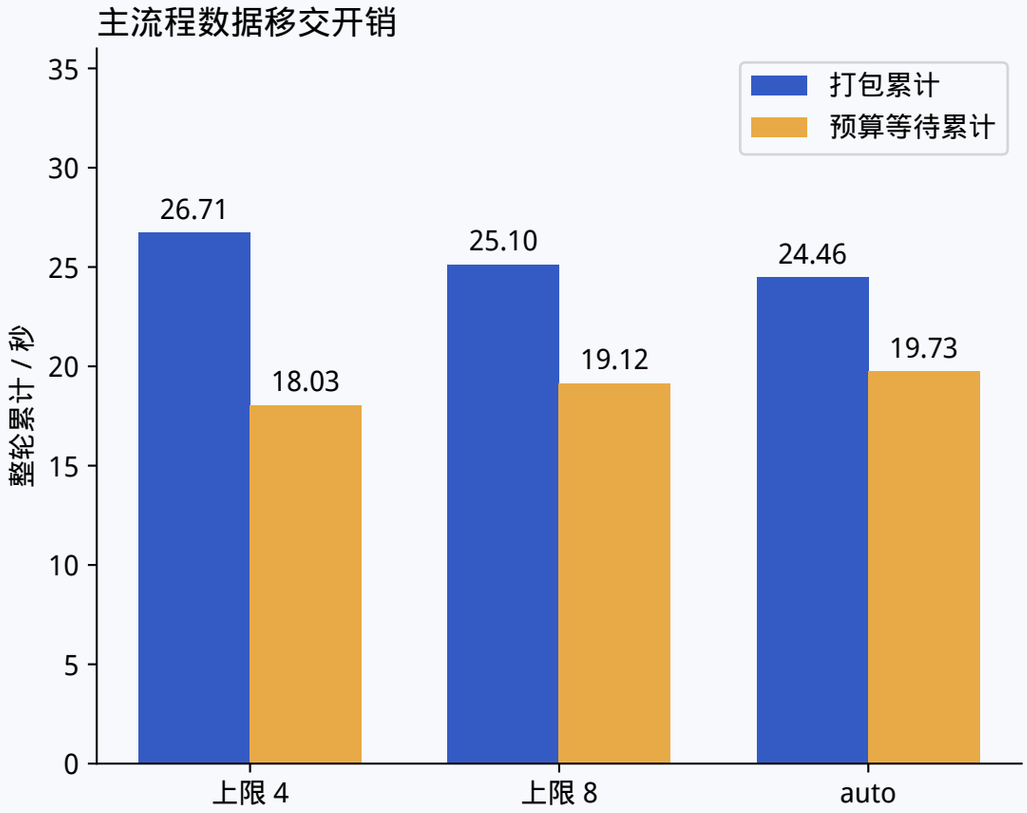
收益主要体现在典型调用还是慢窗口

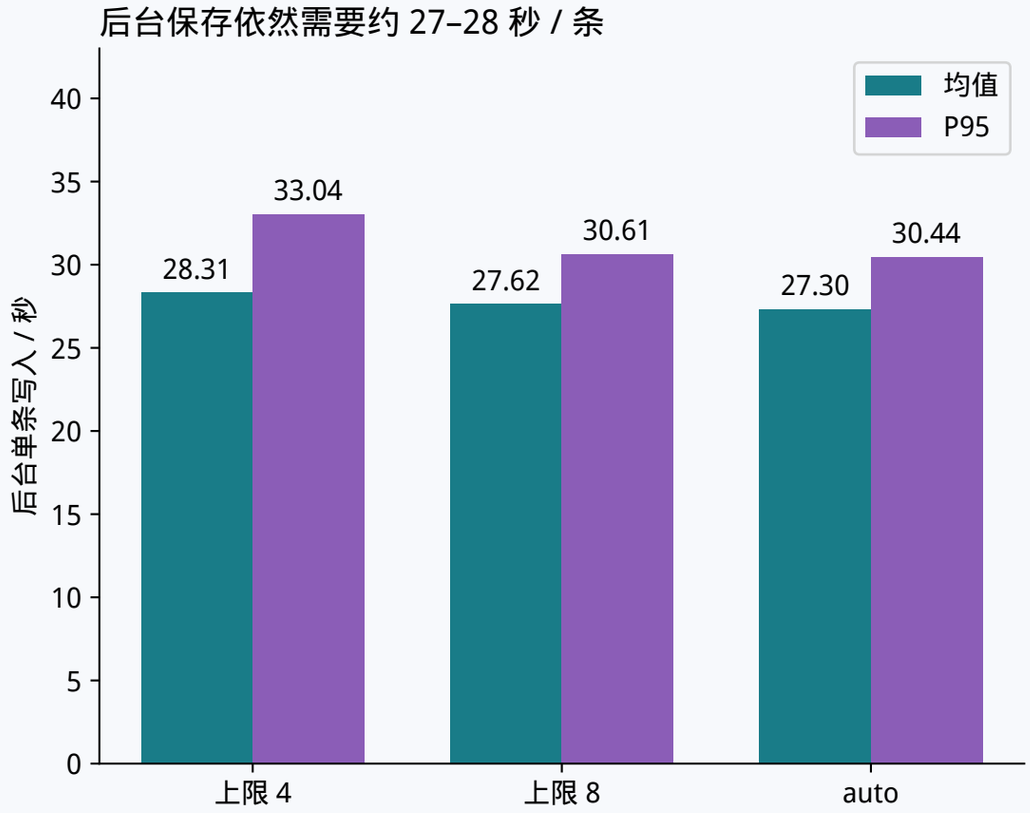
怎么做的	将连续 100 次 execute 调用组成窗口，计算每个完整窗口的平均延迟，再比较中位数和 P95。v1 每组有 225 个完整窗口，v2 有 216 个；不满窗口的尾部不混入同一口径。
这样做的意义	平均值会掩盖偶发阻塞。窗口分布可以观察生成是否经常被保存等慢操作拖住，并帮助判断优化改善的是稳定工作节奏还是极少数尖峰。
当前结论	历史 v1 的典型窗口均值约 108–114 毫秒；同步的慢窗口 P95 约 944 毫秒，auto 约 187 毫秒。差异提示异步保存主要缓解了较长停顿，但具体原因仍需内部计时对应。
读图与术语	这里 P95 是“100 次调用平均延迟”的第 95 百分位，不是单次动作延迟的 P95，也不表示整批 95% 时间。



未归类时间能否直接叫作渲染耗时

怎么做的	用每个 episode 总时长减去已记录阶段，核对计时覆盖范围，并将非负剩余单列为未归类差额。保持原始阶段数据不变，避免为了补齐总数而人为指定差额来源。
这样做的意义	计时覆盖检查可以发现遗漏环节，但只有差额大小还不能判断它属于 CPU、GPU、渲染、调度或其他等待。先把未知部分显示出来，才能决定下一轮需要加在哪里计时。
当前结论	历史 v2 约有 61.86 秒/episode 未归类，说明原计时不足以解释全部经过时间。新增 v1 插桩揭示了 v1 内部构成，却不能直接替历史 v2 差额命名；v2 仍需独立插桩。
读图与术语	图中 execute 已含等待；阶段覆盖率反映计时边界，并不是资源利用率。





右图每组 16 个保存事件；后台并发耗时不能与左图相加为墙钟时间。

异步保存仍然付出了哪些成本

怎么做的

分别记录生产者打包、提交及预算等待，与 writer 后台压缩写入的耗时。前者发生在生成侧，后者可能由多个进程同时执行，也可能与后续轨迹生成重叠。

这样做的意义

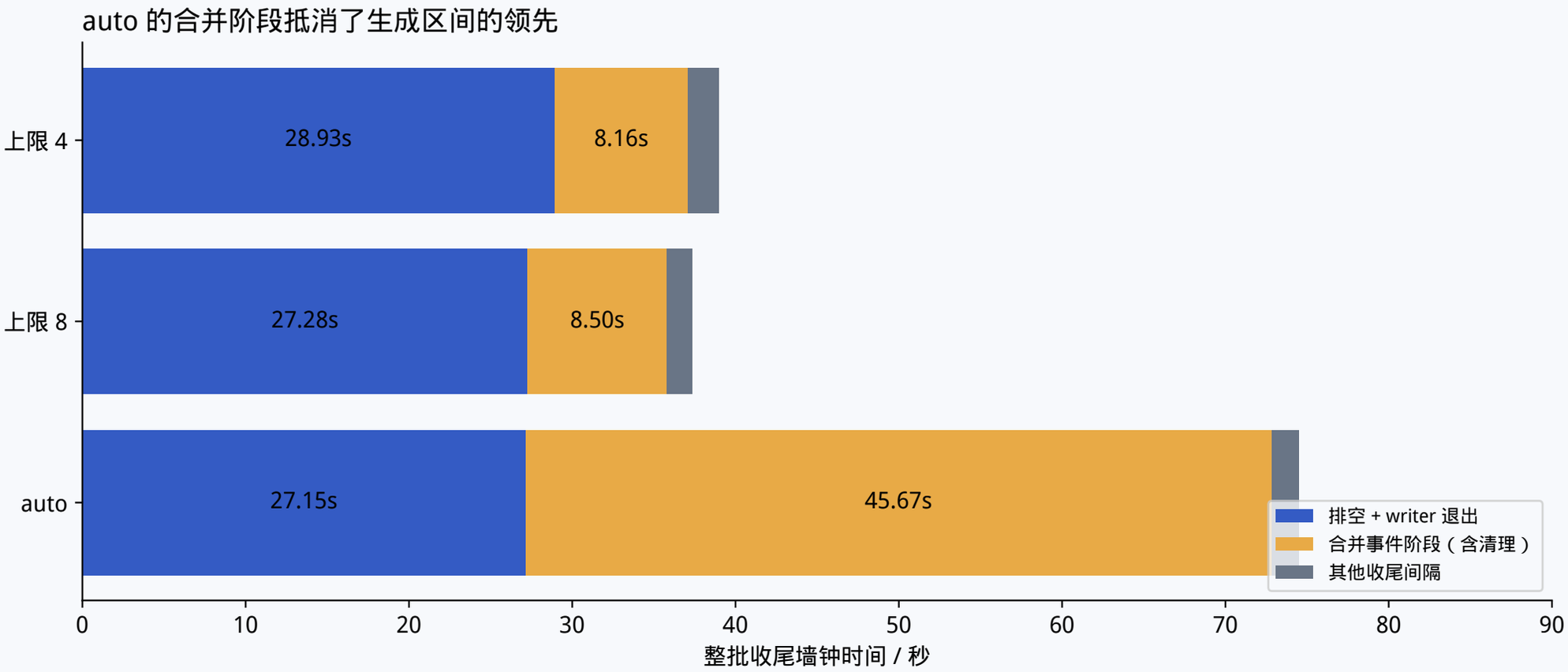
异步不是把保存工作删掉，而是改变它发生的位置和时机。拆开这些成本，才能判断继续优化应面向主流程打包、积压控制，还是后台压缩与存储。

当前结论

历史后台单任务写入仍约 27–28 秒；生成侧提交变短可以与此同时成立。两侧累计时长不能直接相加成最终墙钟时间，预算等待也说明保存能力和内存容量仍会约束生成。

读图与术语

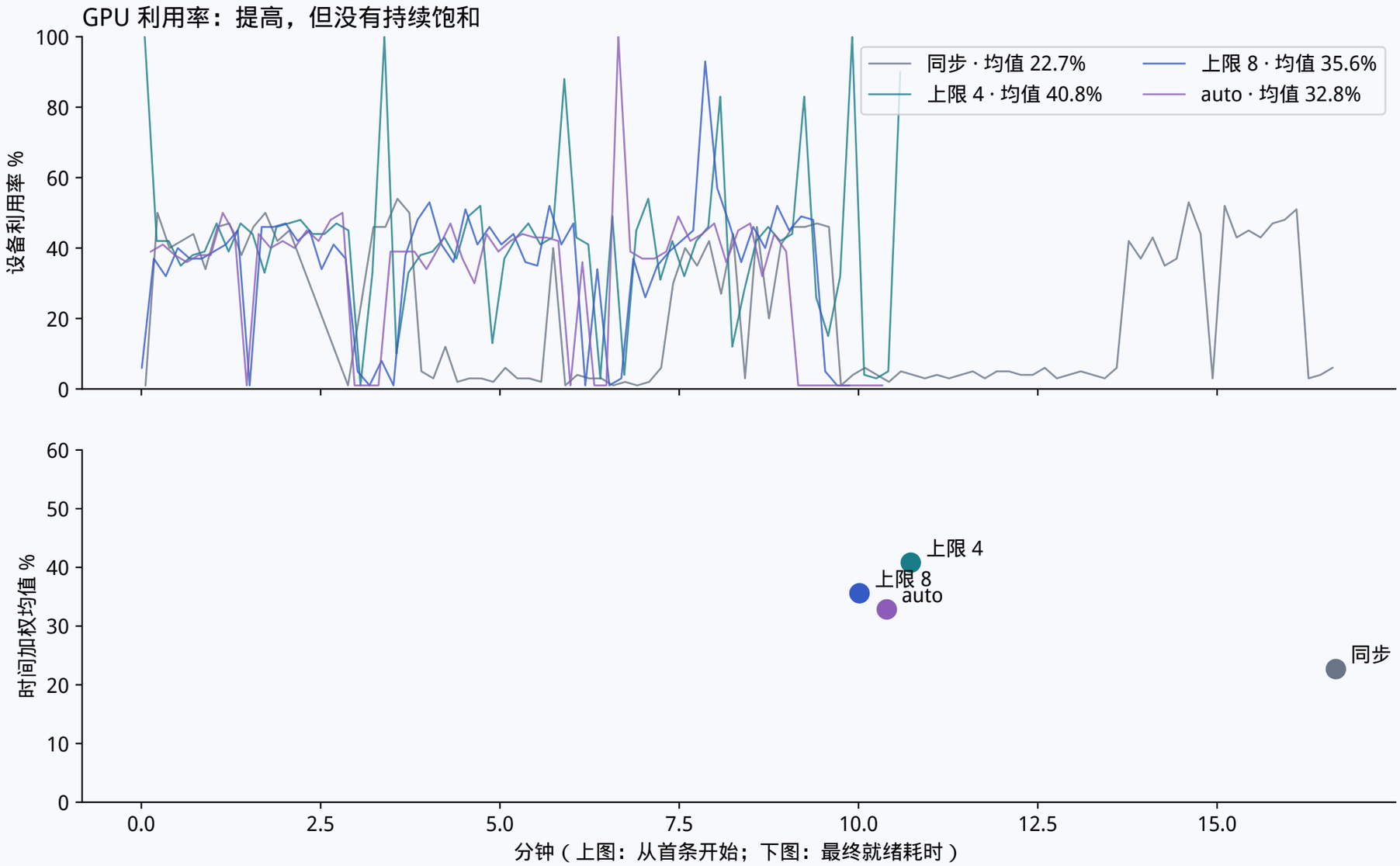
背压指下游处理不及时时，预算或队列限制让上游暂停；这是限制积压的机制，不等于写入失败。



区间：最后 episode 完成 → output_ready。其他间隔为差值，并非独立埋点；同步收尾仅 1.68 秒。

把收尾拆成排空与合并

怎么做的	在最后一个已完成 episode 之后，继续记录剩余保存任务排空、HDF5 分片合并及关闭的时刻。通过这些整批边界区分“还在写”与“已经写完但最终文件尚未合成”。
这样做的意义	不同尾段原因需要不同优化：写入积压应检查保存能力和调度，合并较慢应检查数据复制与文件布局。只增加 writer 未必能缩短合并阶段。
当前结论	历史 auto 的较长尾段需要结合本轮积压与合并记录解释。当前能确认最终就绪必须包含这些工作，但尚未通过重复试验确认其稳定来源，不能据一次尾段给配置定性。
读图与术语	drain 指等待在途保存任务全部完成；merge 指将各 writer 的分片整理成最终输出。



均值采用首条开始至 output_ready 内有效采样的左保持时间加权；设备级指标包含其他进程。

GPU 利用率能解释什么

怎么做的	读取整轮资源采样中的设备 GPU utilization，按同一运行窗口汇总。历史 v1 同步、cap4、cap8、auto 的均值分别约为 22.7%、40.8%、35.6%、32.8%，并结合最终吞吐比较。
这样做的意义	GPU 利用率能提示设备获得工作的持续程度，但整轮还包含规划前处理、录制、保存和收尾。它适合发现供给不连续的线索，不能单独回答哪一个内核最慢。
当前结论	异步组平均 GPU 利用率高于同步，和生成受阻减少的机制一致；均值最高的配置却未必最终吞吐最高。当前指标不提供 SM 占用率、物理与渲染重叠，或纯 GPU 计算耗时。
读图与术语	设备利用率是采样窗口内 GPU 活动指标；SM 占用率是另一类内核执行指标，两者不能互换。

GPU 工作提交受主线程闭环约束：保存并行没有拆开整条路径

等齐动作 → 批量 step	主线程仍执行什么	对应源码位置
各环境准备动作 ↓ put / queue.join 等待队列达到 num_envs ↓ env.step(actions) ↓ task_done 各协程继续下一步	同步规划 plan_single 成功标量 item / bool 观测 GPU → CPU 录制与共享内存 pack 预算不足时等待回执 这些路径可能推迟下一次提交	env_loop：250–275 行 waypoint：333–336 行 planner：1312–1314 行 cameras：54–55 行 export_pool：296–314 行 源码跳转：gpu-analysis.html

GPU 为什么可能等主线程提供下一批工作

怎么做的	沿代码整理闭环：环境任务准备动作，主循环凑齐后批量步进，再更新观测，随后准备下一批动作。主机上的同步规划调用、状态读取、图像复制和保存打包都可能出现在闭环中。
这样做的意义	即使数值计算在 GPU 上并行，下一批输入仍需由主机组织。如果控制流停在 CPU 工作或同步读取上，设备可能得不到连续任务，因此加速不只取决于增加 GPU 算力。
当前结论	异步 writer 移走了闭环中的部分保存工作；相机位姿刷新、物理推进和渲染读取仍留在生成路径。该图说明依赖关系，实际空闲区间和引擎重叠需要 CPU/CUDA/图形时间线测量。
读图与术语	主线程是组织仿真与录制的 CPU 执行线程；同一线程中的同步工作会影响后续 GPU 工作的提交时刻。

历史相机路径与新增配对实测：重复计算需要按状态版本复用

每个 RGB term

update_mounted_camera_poses
循环更新三个相机
↓
读取一组相机 RGB
↓ .to(cpu, copy=True)
CPU 图像副本
↓ observation term.clone()

一次 step 的条件计数

三路 RGB term 启用
且 recorder active
且加载当前 RL step

录制前 compute
重置 / 事件后 compute
→ 六次批量 RGB 回传调用
→ 十八次 set_world_poses

优化须保留语义

相机刷新可评估集中处理
CPU 回传可评估有效缓存

reset / 事件后可能已变状态
不能直接删第二次 compute

调用数不等于渲染次数
需要实测计数与设备追踪

一个控制步为什么出现多次相机操作

怎么做的	源码路径显示普通批量控制步有两次 observation.compute，每次访问三个相机观测项；每项位姿更新又遍历三台相机。新增配对计数核对到每步约 6 次整组位姿刷新、18 次 set_world_poses、6 次 RGB CPU 复制，实际渲染约 3 次。
这样做的意义	相机相关调用并非一一对应真正渲染。区分位姿提交、图像复制和实际渲染，可以优先查找重复主机工作，避免把所有相机调用都当作 GPU 渲染。
当前结论	计数支持检查同状态下的位姿与副本复用；是否能安全复用还要验证 reset、事件、相机变化及观测历史。不能直接删除第二次 observation.compute，也不能把计数减少预先当作等比例加速。
读图与术语	普通步的计数不含额外 reset 开销；“刷新整组相机”与“更新一台相机位姿”是不同计数单位。

先定位设备空隙的来源，再评估批量大小和并行策略

优先验证

1. 加载路径 / hash / 调用数
2. 观测与 RGB 复制时间线
3. 每环境动作就绪时刻
4. bool / item 与规划区间
5. pack 与预算等待

CPU 区间需关联 CUDA 时间线

分开做消融

相机刷新与缓存复用
成功判定批量计算 / 读回
规划与 FK 批处理
共享内存打包调度

单项改变，检查输出一致性
诊断追踪与吞吐运行分开

已有证据与边界

v1 调用数一致，长尾下降
尚无各段 GPU 耗时占比

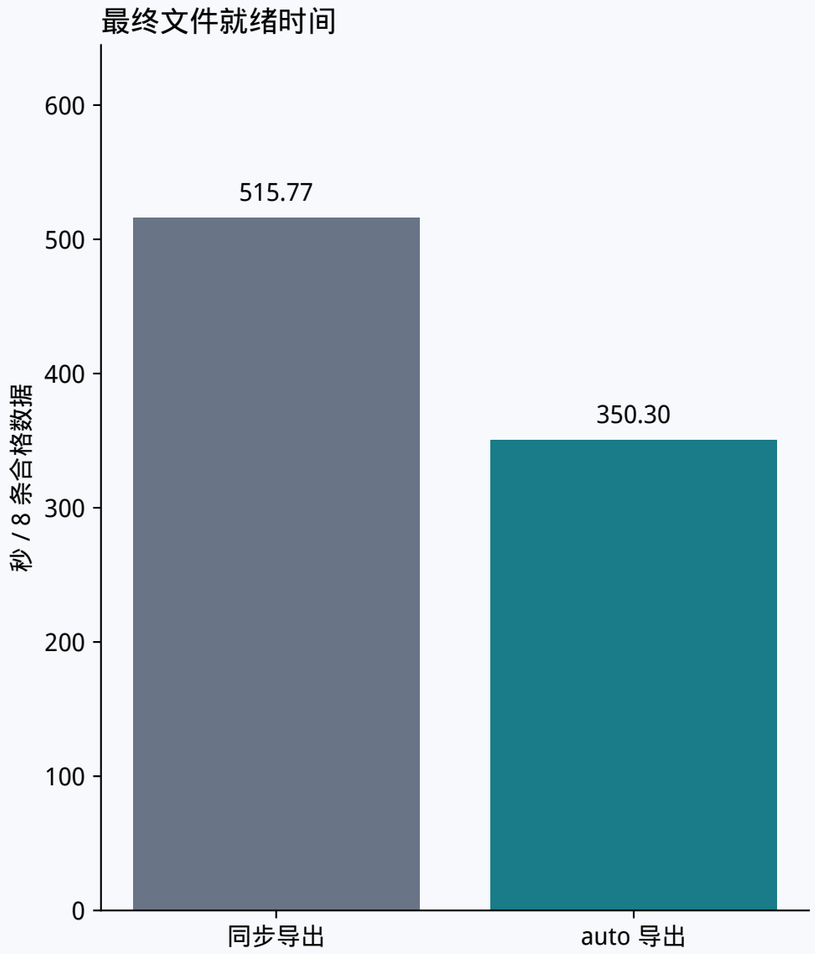
利用率不是 SM occupancy
整批均值还包含收尾阶段

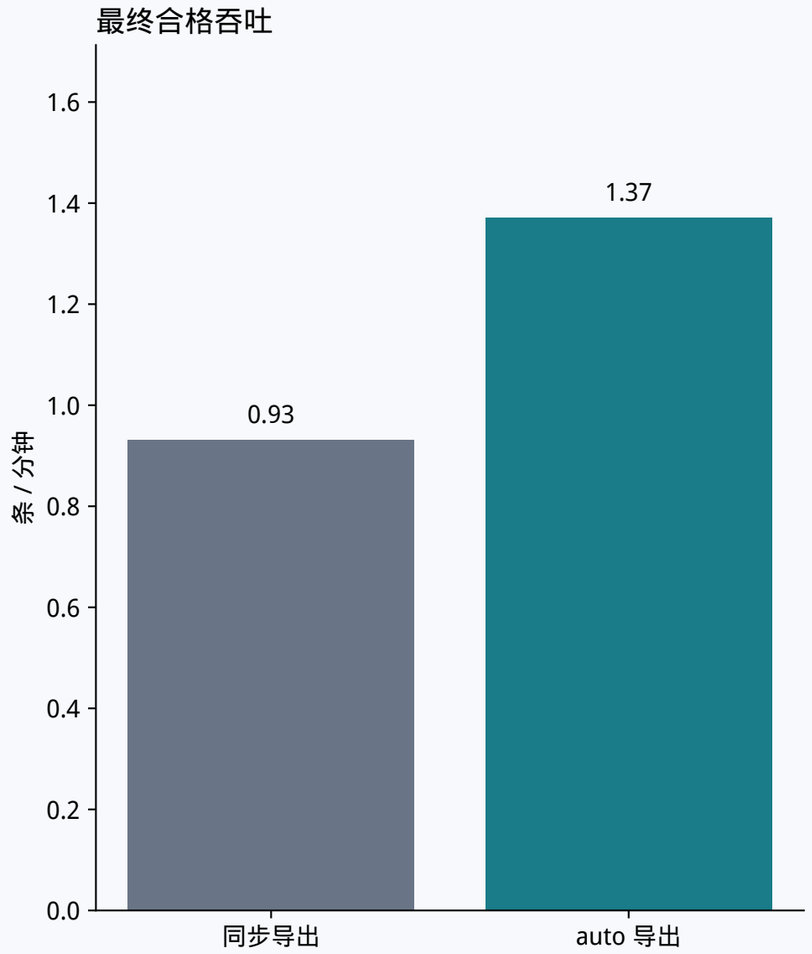
最终评价：合格产出 / 时间
不以利用率越高越好为目标

如何把代码推测变成运行证据

怎么做的	先核对实际加载文件和代码哈希，确认运行路径；再记录调用次数与主机耗时，定位重复工作；随后使用关联 CPU、CUDA 和图形事件的性能追踪，最后通过只改变一个因素的实验验证收益。
这样做的意义	源码说明可能执行什么，计数说明这次执行了多少，时间线说明在哪里等待，消融实验说明改动是否真正改善结果。把这几层串起来，可以减少依据函数名猜测瓶颈。
当前结论	目前已取得冻结代码、真实配对、调用计数及主线程内部归类；尚未取得证明 GPU 引擎实际重叠的追踪。下一步优先验证相机复用，再针对渲染与物理做设备追踪。
读图与术语	消融实验指每次只打开一个改动，并与相同基线比较；性能追踪本身有开销，应与正式吞吐测量分开运行。

新增独立配对：主机分段计时补齐内部归因，历史 8 环境继续单列





共同条件与检查

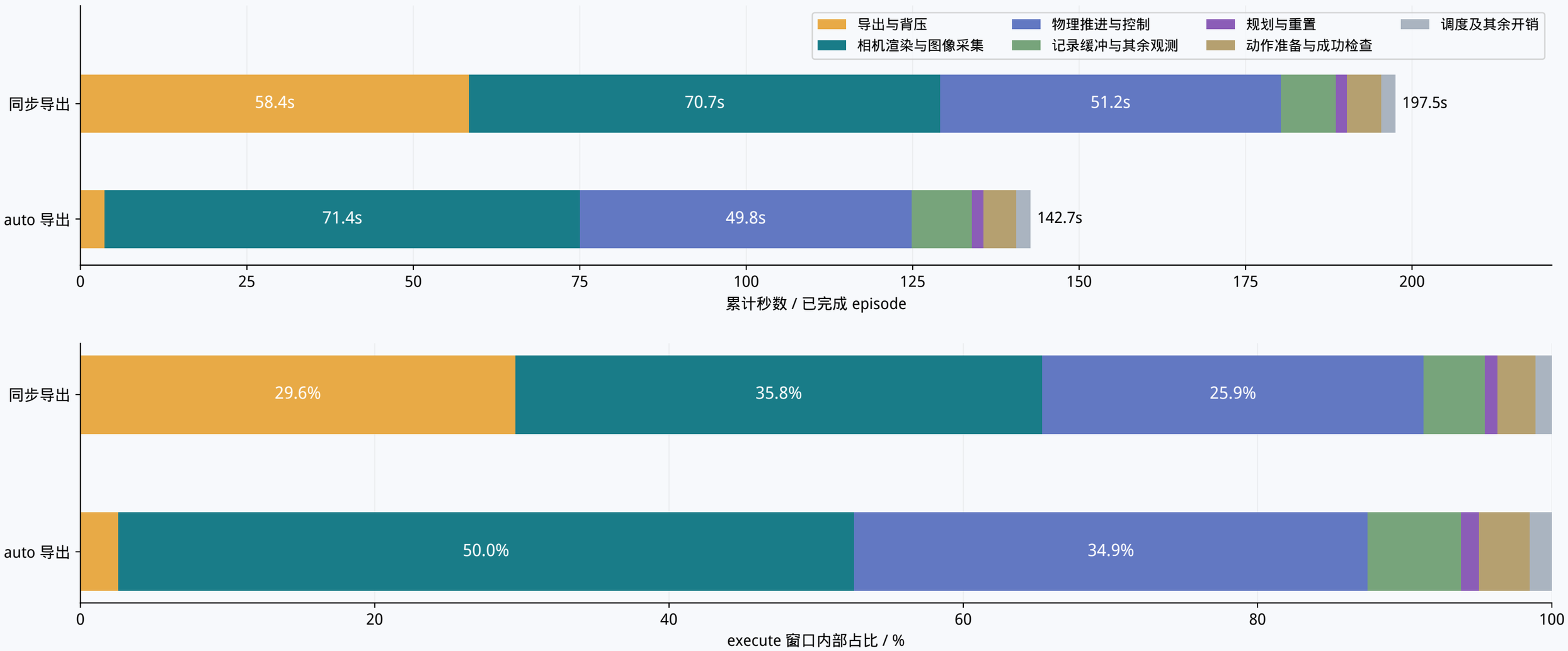
同一 RTX 5090 / cuRobo v1
4 环境，seed 101
各 8 条成功目标
60 个最终保持步

两组严格终态 8/8 通过
结构检查通过
8 组动作序列精确匹配

完整文件吞吐提高 47.2%
新缓存方案尚未实施

新增配对如何保证比较公平

怎么做的	正式配对固定 v1、4 环境、seed 101、8 条保存目标和 60 步末态保持，仅对比原同步保存与改进 auto。两组冻结输入和代码来源，并回读文件，按完整动作数组匹配 8 条数据，同时检查结构及严格末态。
这样做的意义	内部计时需要有可解释的工作量。动作完全匹配、步数一致和质量通过，能排除少执行任务或提前结束等原因；独立保存这组结果，也避免与历史 8 环境数据混用。
当前结论	两组均 8/8 合格，最终就绪由 515.77 秒降至 350.30 秒，吞吐提高 47.2%。8 环境预试曾因内存不足终止，未进入正式结论；本页仅代表这组完成的 4 环境配对。
读图与术语	完整动作按集合匹配允许异步完成顺序变化；这不等于声明所有渲染像素逐位一致，图像复用仍需另做验证。



执行内部的大部分时间花在哪里

怎么做的	用主线程互斥计时区间归类物理、控制、相机、录制和保存工作，再按 episode 的 execute 窗口汇总。每组内部占比的分母都是该组 execute 均值；queue.join 另作为逻辑等待统计，不再次叠加。
这样做的意义	互斥归类避免嵌套函数重复计时，也能把“协程在等”转换成“主线程当时在做什么”。比较绝对秒数与百分比，可以区分真正加速和分母缩小造成的占比变化。
当前结论	execute 均值由 197.53 降至 142.71 秒；保存及背压由 58.41 降至 3.64 秒，解释了几乎全部净下降。相机由 70.72 到 71.41 秒，占比却由 35.80% 升至 50.04%，主要因为保存阻塞减少。
读图与术语	改进后相机约占一半，物理与控制约占 34.90%。这些是主机调用区间归类，不是 GPU 各引擎利用率。

物理推进与控制的分工：主机调度、GPU 工作和阻塞分别理解

环节	在做什么	CPU / GPU 分工	auto execute 内部
物理推进	碰撞、接触、关节与刚体运动 推进机器人与方块的位置 / 速度	GPU PhysX 批量推进 4 环境 主线程调用含提交与等待	36.66s / 25.69%
控制与状态	动作处理、控制器命令 写入仿真与更新状态缓冲	CPU 调度 + GPU 张量运算 不代表全部在 CPU 上计算	13.15s / 9.21%
相机位姿	机器人 link 世界位姿 × 安装变换 设置三路相机的位置和方向	CPU / GPU 混合 线程 CPU / 调用墙钟约 99.3%	36.14s / 25.32%
相机实际渲染	从当前场景生成 RGB 物理状态决定本帧相机与物体位置	GPU RTX 渲染 主线程比值约 45.6%	28.97s / 20.30%
显式 RGB 副本	将图像从 GPU 复制到 CPU 交给 recorder 留存	GPU→CPU 传输 + 主机处理 完成后才能读取该副本	4.16s / 2.91%

物理、控制和相机分别在做什么

怎么做的

物理推进调用 GPU PhysX 更新关节与刚体运动、接触和约束；控制路径组织目标与动作、写入仿真并读取状态。相机路径刷新位姿、调用 RTX 渲染，再将 RGB 图像复制到 CPU 供录制。计时同时对照主线程 CPU 时间。

这样做的意义

一个 API 区间可以同时包含 CPU 提交、设备计算和同步等待。把功能与执行位置讲清，才能决定优化主机调用、重复读取，还是 GPU 内核；不能将整个物理区间都叫作纯 GPU 时间。

当前结论

改进后每 episode 物理 API 约 36.66 秒、控制等约 13.15 秒；位姿刷新 36.14 秒、渲染 API 28.97 秒、RGB 复制 4.16 秒。位姿区间主线程 CPU 时间约占 99.28%，应优先检查主机侧重复刷新。

读图与术语

物理与渲染区间 CPU 时间占比分别约 22.87%、45.58%；其余经过时间可能包含设备或其他等待，不能全部等同于 GPU 计算。

批量环境内部并行；同帧阶段按状态依赖推进



环境维度：同一 GPU 批量推进多个环境，GPU 算子内部存在数据并行。

阶段维度：本帧相机读取依赖物理结果；CPU 消费依赖图像传输完成。

实际每 batch step：4 次物理推进、约 3 次相机渲染、6 批 RGB CPU 副本。

箭头只表示依赖；未采集 GPU 时间线，不能据此断言设备阶段完全串行或已经重叠。

在 GPU 上并行，为什么仍要按步骤等待

怎么做的	4 个环境由一次批量控制步共同推进；配置为 120 Hz 物理、4 次子步，对应 30 Hz 控制。正式运行计数为 2,778 次批量步进、11,112 次物理子步；普通步之后还有位姿、渲染和图像读取。
这样做的意义	环境之间可以数据并行，同一帧内部却有依赖：先得到新物理状态，才能正确更新相机并采集对应图像。GPU 并行并不取消这些因果顺序，CPU 也要组织和读取结果。
当前结论	当前代码和计数确认了批量物理与相机路径，但示意箭头不是设备追踪。是否能跨帧流水化、物理与渲染是否存在可利用的重叠，仍需验证数据对应关系及真实时间线。
读图与术语	物理子步比控制步更细；一条控制动作保持 4 个物理子步。改变子步数量会改变仿真协议，不能直接作为同条件加速。

首选去重方案：保留两次观测，只复用同状态下的昂贵相机结果

当前正常路径

每步两次观测计算
每次读取三路 RGB

每个 RGB term 刷新全相机
6 次全相机刷新
→ 18 次相机位姿设置

6 批 RGB CPU 副本
实际渲染已约为 3 次

待验证的快路径

有效状态版本内
统一刷新三路相机

6 次全相机刷新 → 1 次
18 次位姿设置 → 3 次

同帧 CPU RGB 副本复用
6 批 → 3 批
固定索引和变换也复用

缓存失效与所有权

reset_epoch / state_revision
相机 pose_revision / frame_id

重置与事件后立即失效
保留第二次 compute/history
不接受过期帧或一帧延迟

每路只持有有界单帧
recorder 按环境持有不变

优先尝试复用哪些相机工作

怎么做的	候选方案一是在同一物理状态下，将 6 次整组相机位姿刷新合并为一次；方案二让同状态 RGB 的 CPU 副本供两次观测读取复用，使普通步复制由 6 次降为 3 次。保留两次观测计算及历史更新语义。
这样做的意义	这两项针对已观测的重复主机工作，有机会减少开销而保持动作、帧率和分辨率。缓存必须绑定物理状态、reset、事件与相机状态，状态变化后立即失效。
当前结论	目前这是待实施、待消融的方案。应先做位姿去重，再单独测 RGB 复用和组合；只有逐帧对应、历史缓冲、末态与内存检查通过，才可将复用视为保持任务语义的优化。
读图与术语	缓存副本需保证读取期间不被后续帧覆盖，缓冲数量有界；仅按控制步编号判断相同状态可能漏掉 reset 或相机变化。

按实测占比选择优化，以最终合格吞吐评价

顺序	方案	实测依据	正确性与收益边界
P0	位姿刷新去重 复用固定索引 / 变换	25.32% 主线程占用明显	保持 reset / 事件 / 位姿失效语义
P1	同状态 RGB CPU 副本复用 保留完整观测计算	2.91% 显式复制 未含任意观测的全部成本	帧序、history、不可变性、内存有界
P2	设备时间线后定位 renderer 实际跨相机提交和状态同步	20.30% 渲染调用 不等于隔离 GPU kernel 时间	本版本 TiledCamera 已是别名 不能只换类名期待加速
P3	定位物理提交 / 求解 / 等待 批量成功判定与小张量操作	物理推进 25.69% 控制 / 更新另 9.21%	先保持 dt、子步、接触与成功阈值
后置	有界 pinned 缓冲 + 异步 D2H 打包与规划/FK 分别消融	复制 2.91% auto 导出相关 2.55%	事件完成后 CPU 才读；禁止提前复用 不盲目增加环境数或 writer

依据耗时和风险安排优化顺序

怎么做的

以改进后 execute 构成为基准，先检查占 25.32% 的位姿刷新与占 2.91% 的 RGB 复制；再追踪占 20.30% 的渲染 API、占 25.69% 的物理 API，以及约 9.21% 的控制与状态路径。

这样做的意义

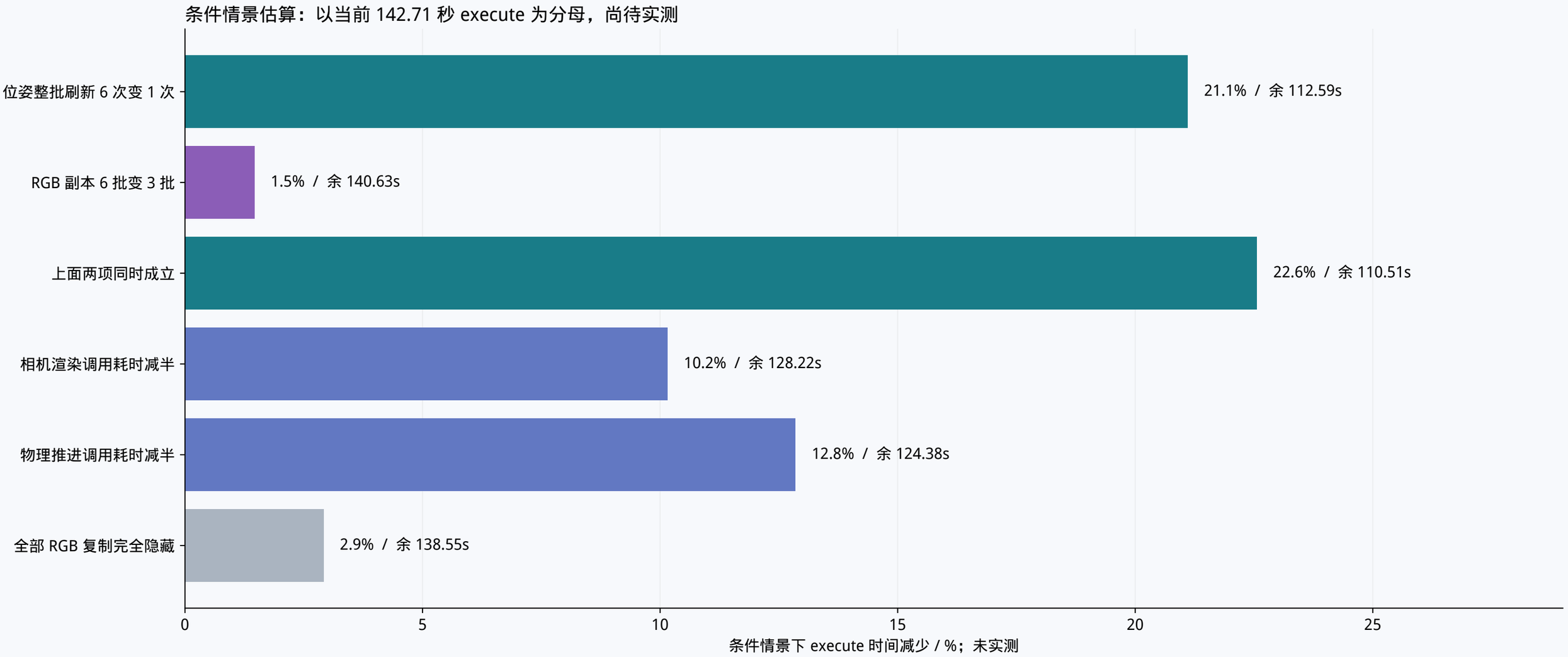
优先级同时取决于占比、可消除的重复量和正确性风险。主机位姿去重有明确重复计数；渲染和物理虽大，却需要设备追踪确认真正瓶颈，不能简单按 API 名称估计收益。

当前结论

先进行相机复用消融，再决定是否投入渲染或物理优化。当前本地 TiledCamera 是 Camera 的弃用别名，仅换类名不会增加新的批量能力；分辨率、采样频率、物理步长和成功阈值变化应另设质量实验。

读图与术语

优先级不是收益承诺。异步 GPU→CPU 复制还需要完成同步与缓冲生命周期保证，当前复制占比较小，应排在位姿去重之后。



如何估计方案的条件收益

怎么做的	从改进后 execute 均值 142.706 秒出发，假设其余成本不变，按候选部分的可减少比例做算术扣减。例如位姿刷新耗时 36.136 秒，若 6 次降为 1 次且成本线性，则减少其中 5/6。
这样做的意义	条件估算用于判断实验是否值得做，也暴露收益依赖哪些假设。它不是预测器：调用合并可能存在固定开销，缓存会增加管理成本，最终排空和合并也未必同步缩短。
当前结论	该假设下，位姿去重使 execute 降至约 112.59 秒，减少 21.1%；再将 RGB 复制减半，降至约 110.51 秒，减少 22.6%。这些均未实测，不能换写成最终吞吐提高 21.1% 或 22.6%。
读图与术语	读图先看假设，再看扣减量。百分比的分母是 execute 均值，整批 output_ready 还有不同范围的工作与重叠。

下一轮：输出等价通过之后，才比较最终合格吞吐

单项消融与重复

A0 现有 auto
A1 只去重位姿刷新
A2 只复用 CPU RGB
A3 两项组合

4 环境、8 条成功目标
同 seed / 分辨率 / 压缩
至少 3 对交替顺序重复

逐帧正确性

完整 actions 与严格终态
frame_id 与 reset_epoch
重置 / 事件后的第一帧
两次 compute 的 history
CPU 帧不可变与归属
GPU / CPU 缓冲有界

图像容差来自基线重复

计时与设备诊断

记录主循环与最终就绪
分段闭合 / 内存峰值
主线程 CPU / 设备时间线

NVTX + CUDA / 图形追踪
先核对本地工具兼容性

profiler 与吞吐运行分开
以合格条数 / 分钟判断

下一轮怎样验证相机方案

怎么做的	设置 A0 现有 auto、A1 仅位姿去重、A2 仅 RGB 副本复用、A3 两项组合。固定 4 环境、8 条目标、seed 101 及原任务配置，计划至少 3 组配对并交替运行顺序；正式计时与性能追踪分开。
这样做的意义	单独打开每个改动，可以判断收益来自哪一部分，以及组合后是否互相影响。交替和重复能减轻缓存、系统负载与运行顺序对结果的影响。
当前结论	这是后续实验协议，尚无 A1–A3 实测结果。验收需同时满足动作与步数匹配、严格末态和文件结构通过、逐帧图像及观测历史正确、reset 后缓存失效、内存有界，并比较最终就绪时间。
读图与术语	任何一项语义或质量检查失败，都需要修正方案后重跑；不能仅凭某个 API 计时下降宣布生产加速。

从观察到解释：每一步结论都对应证据

已观察到

v1 异步最终就绪更早
执行 / 等待阶段均值下降
后台每条仍保存约 27–28 秒
末态检查均为 16/16

这是本轮完整系统的结果

与代码一致的解释

保存转入 CPU 子进程
生成与保存可在时间上重叠
主流程减少压缩与写入负担
仍承担打包和预算等待

机制解释不等于贡献分解

仍待验证

每项改动各贡献多少？
重复运行排名是否稳定？
GPU kernel 是否相互重叠？

需要消融、重复与设备追踪
不能只从 GPU 利用率推断

哪些是观测，哪些是机制解释

怎么做的	把证据分成三层：运行日志给出时间与质量结果；冻结源码和新增计数说明调用路径；独立消融用来验证单项改动的因果贡献。每项结论对应到它实际拥有的证据层。
这样做的意义	总时间下降能证明这次组合改造有效，却不能自动给每个代码改动分配收益。区分观测与解释，可以让后续方案直接针对尚未确认的环节设计实验。
当前结论	v1 异步保存收益已有历史主实验和新增配对支持；内部归类显示保存阻塞下降是主要变化。相机去重、RGB 复用以及各项内存改动的独立贡献仍待消融，GPU 引擎重叠仍待追踪。
读图与术语	“与某机制一致”表示现象支持该解释；“独立贡献已验证”还要求只改变该因素的对照结果。

cuRobo 库级变化 ≠ 本次 AutoData 全部启用并验证

维度	库级能力变化	本次实验覆盖
GPU 并行	v1 已有多初值并行求解 v2 继续扩展 GPU 计算	每环境规划器；未实现跨环境批量请求 不把 CPU writer 当作 GPU 并行升级
轨迹优化	v1 已有平滑性目标 v2 强调 B-spline 与动力学约束	读取 v2 插值轨迹 未验证本任务力矩约束激活与满足
场景感知	v2 增强 GPU 原生 深度融合 TSDF / ESDF	从 USD 解析碰撞几何 没有深度融合建图对照
高自由度计算	v2 强调拓扑感知运动学 可微逆动力学与自碰撞计算	OpenArm 左臂任务 没有全身机器人性能对照
性能结论	官方结果对应其测试任务 不能直接转成生产吞吐收益	v2 完整保存 16 条、末态通过 15/16 跨日期单轮，尚未确立性能优势

库的能力与本项目启用的能力

怎么做的　将 cuRobo v1/v2 的库级接口与能力说明，对照本项目冻结适配器实际构造的模型、目标、场景和轨迹。逐项检查 GPU 求解、动力学约束、深度建图和高自由度能力是否在这次任务中启用并验证。

这样做的意义　官方库能做什么，与本项目当前使用了什么，是不同范围的问题。只有调用路径和实验覆盖一致，才能把某项能力与实际生产结果联系起来。

当前结论　本次完成了 OpenArm 任务的 v2 规划适配，场景仍来自 USD 几何；未做深度融合建图、全身机器人或力矩约束专项对照。库级性能数字不能直接换算成本项目生产吞吐收益。

读图与术语　USD 是场景描述格式；FK 是正向运动学，即由关节状态计算末端位姿。v1 本身也已有 GPU 内部并行求解。

接口迁移：目标类型、参数顺序、配置传递和轨迹语义均需适配

v1：当前适配器调用

配置构造
MotionGenConfig.load_from_robot_config(...)
MotionGenPlanConfig(...)

目标与调用
Pose
plan_single(start_state, goal_pose, plan_config)

轨迹结果
优先 optimized_plan，空时取插值
get_full_js → get_ordered_joint_state

v2：实验冻结适配器调用

配置构造
MotionPlannerCfg.create(...)
robot / scene_model / device_cfg

目标与调用
GoalToolPose.from_poses(...)
plan_pose(goals, start_state, max_attempts=...)

轨迹结果
get_interpolated_plan()
reorder(joint_names) → FK → 世界系位姿

上层保留 update_world_and_plan_motion(...) → bool；enable_retiming 默认值仍有差异，详见正文。

---

v1 到 v2 为什么需要适配层

怎么做的	v1 用 MotionGenConfig、Pose 和 plan_single 构造并求解目标；v2 冻结适配器使用 MotionPlannerCfg、GoalToolPose 和 plan_pose。结果经插值、关节顺序转换及 FK 后，再交给原任务执行路径。
这样做的意义	接口迁移不只是改函数名。目标表达、参数顺序、轨迹结果和坐标系若转换错误，代码可能正常返回，却让机器人执行错误的运动；适配层用于维持上层任务约定。
当前结论	当前已打通两种接口与上层布尔返回约定，但部分默认值仍需核对，例如 enable_retiming。接口可运行说明集成完成，并不证明所得轨迹更快、更平滑或更可靠。
读图与术语	关节重排让规划结果按控制器期望的关节顺序输出；插值将规划轨迹转换为执行需要的离散序列。

cuRobo v2：保留 AutoData 任务契约，适配模型、规划与轨迹接口

输入适配	规划与接触	输出适配
原机器人配置深拷贝 ee_link → tool_frames retract_config → default_joint_position	MotionPlanner.plan_pose GoalToolPose 目标	get_interpolated_plan 读取稠密执行轨迹
目标 / 物体 → 基座系 开闭夹爪参数原地更新	搬运时附着物体 接触阶段临时调整碰撞 finally 恢复 / detach	按 joint_names 重排 可选关节距离重采样 FK → 世界系末端位姿

8 个环境各有规划对象；不把样条控制点直接执行，也不把规划成功等同于最终数据合格。

迁移时要保持哪些语义一致

怎么做的	适配器将机器人配置、碰撞场景、目标工具位姿和附着物信息映射到 v2；求解后处理关节顺序、轨迹插值，并将末端位姿转换回任务使用的世界坐标系。
这样做的意义	机器人和场景的表示决定碰撞检查与目标约束含义。保持这些语义一致，才能把后续差异归于规划后端，而不是坐标系、关节或附着关系转换错误。
当前结论	已实现可运行的迁移路径并产出完整数据，仍需围绕末态失败、默认参数差异和相同任务重复对照继续验证。适配层完成是性能比较的前提，尚不是性能优势的结论。
读图与术语	附着物指被机器人夹持、随运动更新的物体；世界坐标系是任务、相机和场景共同参照的坐标系。

v2 补跑设计：补齐输出链，再讨论版本差异

为什么重跑

旧记录缺最终就绪与校验链
部分分片已不在原路径

旧记录保留为不完整证据
新输出使用独立目录
不覆盖旧实验状态

怎样运行与验收

冻结 launcher 与保留环境
同源输入，8 环境，seed 101
目标 16 条记录成功

生成 → 保存 → 排空 / 合并
最终文件读回 + 原阈值检查

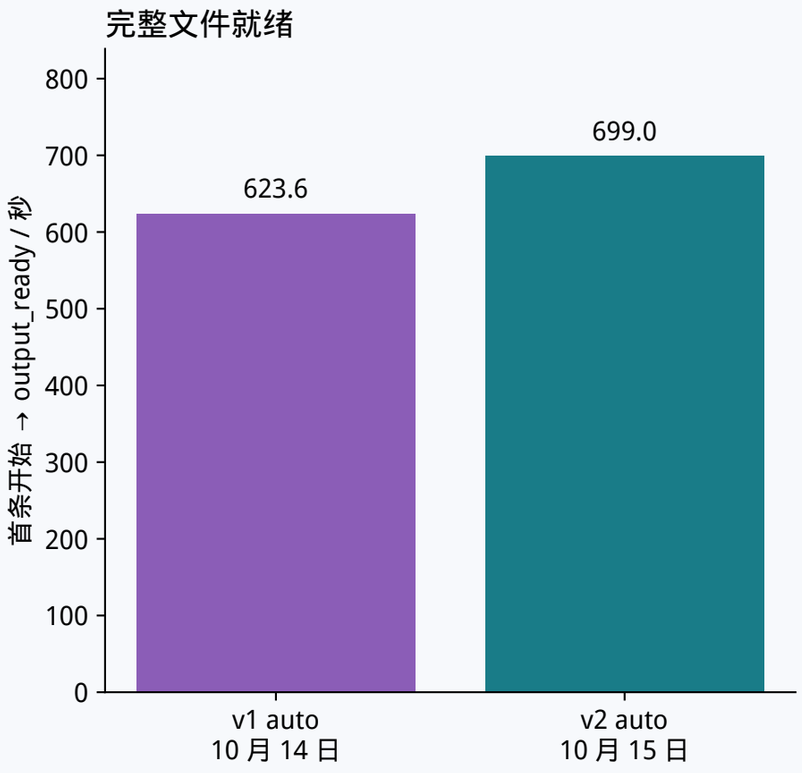
哪些条件未控制

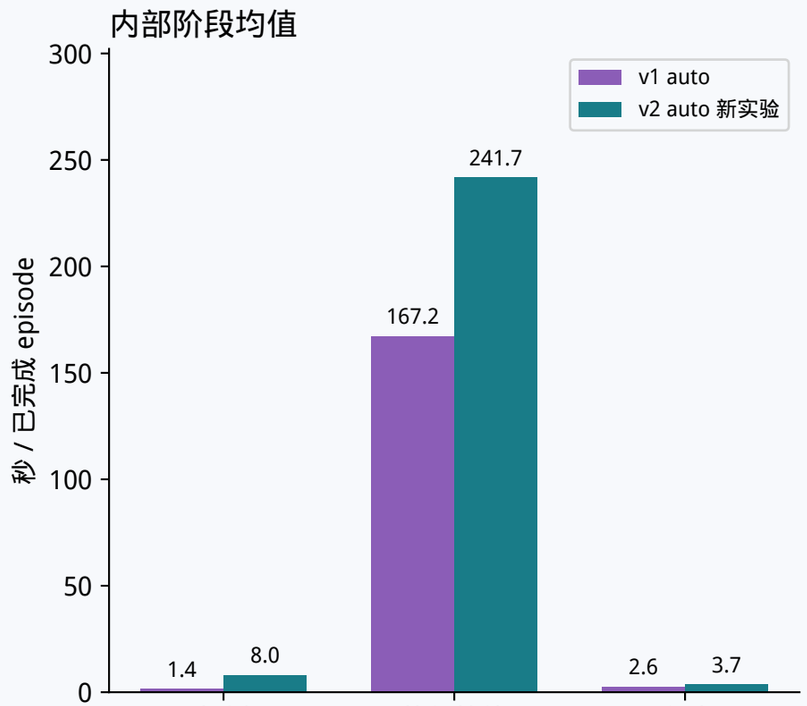
v1 与 v2 测试日期不同
缓存、热身与系统状态不同
完成 episode 分母不同

没有同日交错重复
因此不隔离纯后端因果效应

v2 重跑采用怎样的验收流程

怎么做的	次日 v2 使用冻结启动器、8 环境、seed 101、16 条保存目标，沿用异步 auto 保存和原成功阈值。整轮结束后回读 HDF5，检查结构，并用严格物理末态验证器独立检查每条保存数据。
这样做的意义	运行完成只表示程序走到结束；独立回读和末态检查才能确认输出可用。保留原阈值也避免通过放宽成功条件让新后端看起来更快或更成功。
当前结论	该协议给出了 v2 的完整运行证据，但与历史 v1 测试不在同一天，也只有一轮。结果适合描述这次接入状态；建立稳定性能差异需要同条件、交替重复的配对实验。
读图与术语	文件结构检查关注字段、形状和条数；严格末态检查关注最后的物理状态，两种通过条件缺一不可。





新实验检查（末态未全通过）

生成成功：16 / 16
最终保存：16 条
末态通过：15 / 16
实际 writer：4
记录成功：1.373 条 / 分钟
末态合格：1.287 条 / 分钟

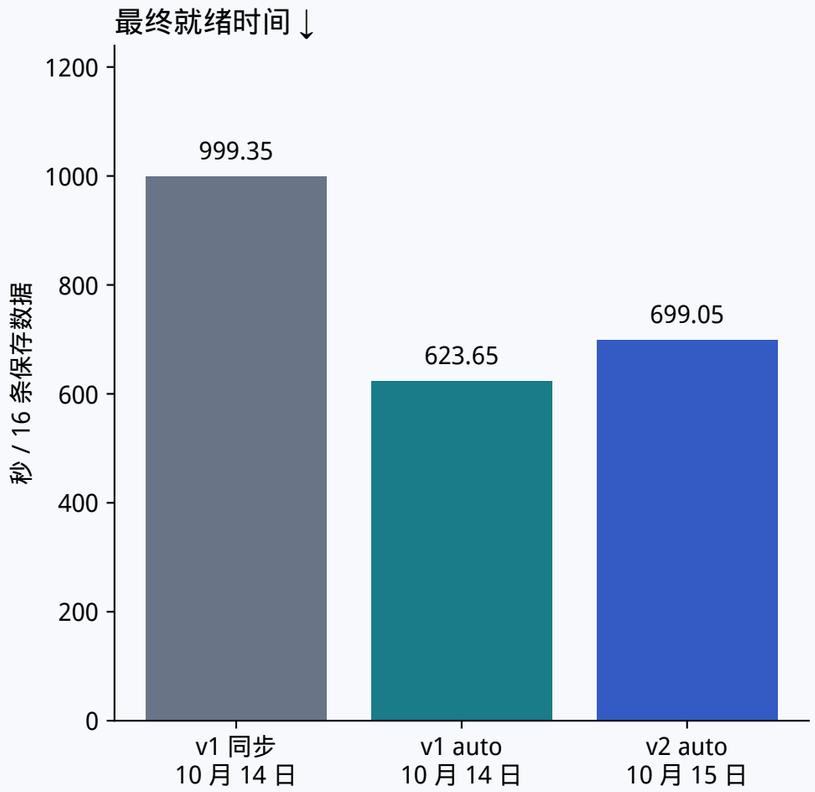
10 月 14 日　　10 月 15 日

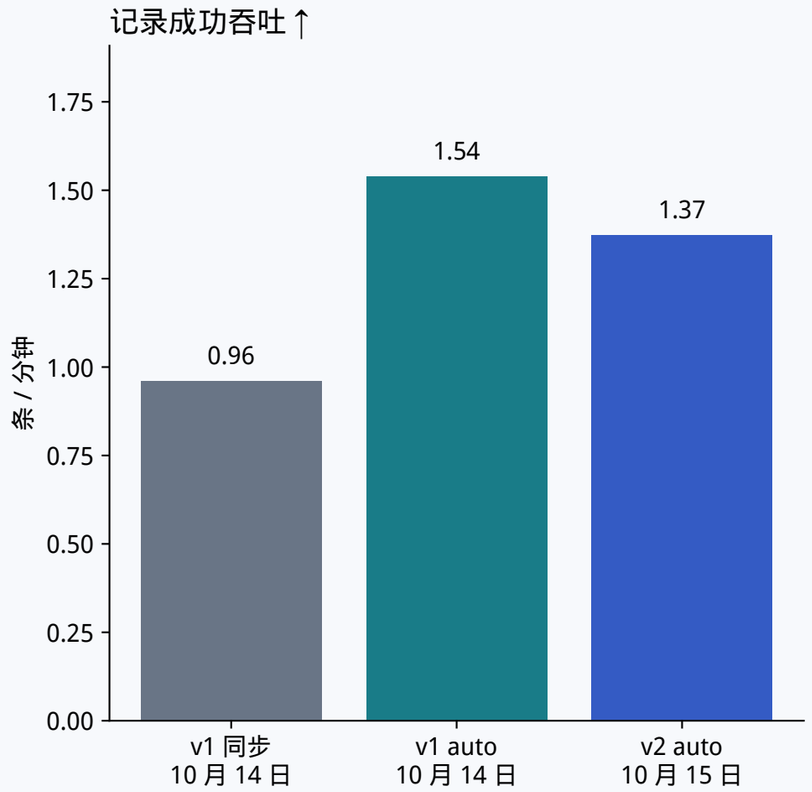
跨日期单轮观测，缓存 / 热身 / 系统状态未控制一致；不能将差值单独归因于版本升级。

16 条保存完成为何仍有质量问题

怎么做的	分别报告最终输出时间、保存条数和严格末态通过数。v2 整批最终就绪约 699.05 秒，16 条均保存并可回读，但其中 demo2 未通过严格末态检查，合格数为 15 条。
这样做的意义	任务内部成功信号与保存后的严格物理检查可能不同。把这两个结果并列，才能避免用“保存 16 条”代表“16 条全部合格”，也为失败轨迹复查提供明确对象。
当前结论	本次 v2 集成运行完成，严格末态为 15/16。记录成功吞吐使用 16 作分子，末态合格吞吐应使用 15；现有结果不支持宣称 v2 在该任务上优于 v1。
读图与术语	未通过项 demo_2 的末态角速度为 0.8538 rad/s，超过 0.5 rad/s 阈值；位置与线速度合格不能替代角速度条件。应复查保持阶段和适配参数，再重复验证。

实现改变不等于性能优势，需结合完整输出与质量





解释与质量边界

v1 auto 对比同步
耗时减少 37.6%
吞吐提高 60.2%
末态均通过 16/16

v2 auto：末态通过 15/16
完成接口迁移与整轮运行
尚未建立性能优势

把版本改动与完整结果放在一起看

怎么做的	并列历史 v1 同步、v1 auto 与次日 v2 auto 的最终就绪时间、记录成功吞吐和严格末态结果。各组保存目标均为 16 条，但版本、日期和质量通过数在图中分别标注。
这样做的意义	实现变化需要落到结果上检验。时间、产出和质量同时展示，可以防止只选取最有利的某一个数字，也能明确哪个比较支持保存优化、哪个比较仍受实验条件限制。
当前结论	历史 v1 auto 相对同步耗时减少约 37.6%、吞吐提高约 60.2%，两者保存数据均 16/16 合格。v2 为 15/16 合格，且跨日期单轮，当前没有确立规划迁移的性能优势。
读图与术语	新增 4 环境配对的吞吐收益为 47.2%，来自另一组规模和计时条件；它补充证据，不替换本页历史数字。

结论：效率改进、接口迁移与质量验收分别陈述

已观察到的收益

v1 auto 对同步基线
最终就绪耗时减少 37.6%
记录成功吞吐增加 60.2%
末态检查均通过 16/16

本轮整体保存改造有效

已完成的 v2 工作

迁移规划模型和调用接口
正确读取插值轨迹并重排关节
完整生成并保存 16 条
完成输出读回与末态检查

末态通过 15/16，未全合格

不能推出的结论

不能保证 auto 总是最快
不能把吞吐收益当 GPU 加速比
不能声称 v2 已建立性能优势
不能把库全部能力当作已验证

所有结果保留单轮限制

哪些结论可以用于当前决策

怎么做的	综合历史 8 环境生产对照、新增 4 环境内部计时、文件质量检查和 v2 重跑，将结果按已验证、可解释、待实验三个范围整理。对不同规模的数据分别保留其分母和条件。
这样做的意义	研究结论应能指导下一步投入：已经证明有效的路径可以保留，内部热点决定优化顺序，尚未验证的方案则需要明确的实验门槛。
当前结论	当前支持保留 v1 异步保存；其主要实测变化是减少生成路径中的保存及背压。改进后应优先验证相机位姿去重与同状态副本复用。v2 可继续用于迁移研究，暂不作为已证明更优的生产替代。
读图与术语	相机方案的 21.1%/22.6% 是 execute 条件估算，尚无优化实测；实际决策以完整输出、质量与内存结果为准。

下一轮验证：先规定问题和判据，再执行实验

性能稳定性

同日交错重复 v1 / v2
保持输入、种子与计时一致
记录热身和系统状态
报告中位数、波动与合格吞吐

本页为方案，尚未执行

收益来源

分别改变保存策略与 writer 上限
记录实际进程数、预算等待
设备追踪单独做诊断运行
测动作就绪、规划与 kernel

避免追踪开销污染吞吐对照

v2 质量失败

复核 demo_2 末态角速度
检查结束条件与稳定等待
修复另记版本并完整重跑
保留原阈值与本轮失败记录

末态验收与性能比较同时报告

下一轮需要回答哪些问题

怎么做的	保存侧通过同日交替重复确认收益稳定性，并分开测试 writer 和内存改动；生成侧执行相机 A0–A3 消融，采集内部计时；v2 复查失败末态并安排同协议对照，设备追踪另行诊断。
这样做的意义	这些实验分别消除配置排名不稳定、组合改动难归因、相机复用未验证和 v2 质量边界等不确定性。按问题分开设计，比同时更换多项参数更容易得出可行动的结论。
当前结论	优先完成保持任务语义的相机去重，再考虑渲染与物理深度优化。每次正式比较都应报告最终就绪、合格吞吐、质量通过数及内存峰值；当前没有下一轮的新增性能结果。
读图与术语	同日运行也可能受系统负载影响，所以仍需交替和重复。不能将带追踪开销的运行直接与无追踪基线比较。

改进前：生成主流程等待导出



改进后：生成与保存重叠



机制图，无时间比例。质量判定保持独立；异步提交完成不能代替最终文件就绪。

从保存流程理解异步收益

怎么做的	同步流程在一条轨迹结束后由生成主流程完成压缩和写入；异步流程先提交有界队列，生成下一条轨迹时后台继续保存，最后统一排空并合并。图按逻辑顺序画出两条路径。
这样做的意义	生成和保存需要不同资源，允许它们重叠可以减少总等待。只有保留最终收尾，才能判断重叠是否带来完整产出的收益。
当前结论	这张示意图解释流水线机制，实际收益以主线生产实验为准。即使只有一个 writer，也可能通过生成与保存重叠获益；多个 writer 则进一步改变保存侧处理容量。
读图与术语	图中框宽和箭头间距不代表测量时长；最终文件完成仍依赖后台任务与合并全部结束。

保存机制的串并行对照：所有生产配置仍使用 8 个仿真环境

同步保存 · workers=0



同一主流程承担保存等待

异步单 writer



提交后可继续生成；保存侧仍只有一个执行者

异步多 writer / auto



生成与保存重叠；在途内存超预算时等待，最终仍需合并

控制流示意，无耗时比例。异步提交后生成可继续，后台保存同时推进；不是三个版本的实测时间线。

流水线重叠与保存并行有何区别

怎么做的	分别画出同步保存、异步单 writer 和异步多 writer：单 writer 可与生成重叠，但保存任务在该进程内顺序处理；多 writer 可同时处理不同 episode，最后仍由主流程排空和合并。
这样做的意义	将两种机制分开，才能设计有效对照：同步对异步单 writer 检验流水线重叠，单 writer 对多 writer 检验保存并行容量，而不是把所有变化归到“多进程”三个字。
当前结论	主实验验证的是保存改造组合效果，未完整覆盖这两项独立消融。因此当前不能分别报告流水线重叠与多 writer 的因果贡献；图中单 writer 是机制说明，不是新增生产测量。
读图与术语	本附录沿用历史 8 环境生成背景；writer 数量影响保存侧，仿真环境数量没有随之改变。

看并行单位与资源边界，不能只看数量

单进程多环境批量步进

当前生成侧

8 个环境任务协作调度
↓
动作汇合后批量步进
↓
GPU 数值计算

共享事件循环与仿真对象
同步调用 / 动作屏障会等待

CPU 多进程保存

当前保存侧

主进程打包 episode
↓ 共享内存
多个 writer 独立压缩
↓
各写分片 → 排空 / 合并

生成与保存可重叠
竞争 CPU、内存与磁盘

使用 GPU 的多个生产进程

未测试的另一种方案

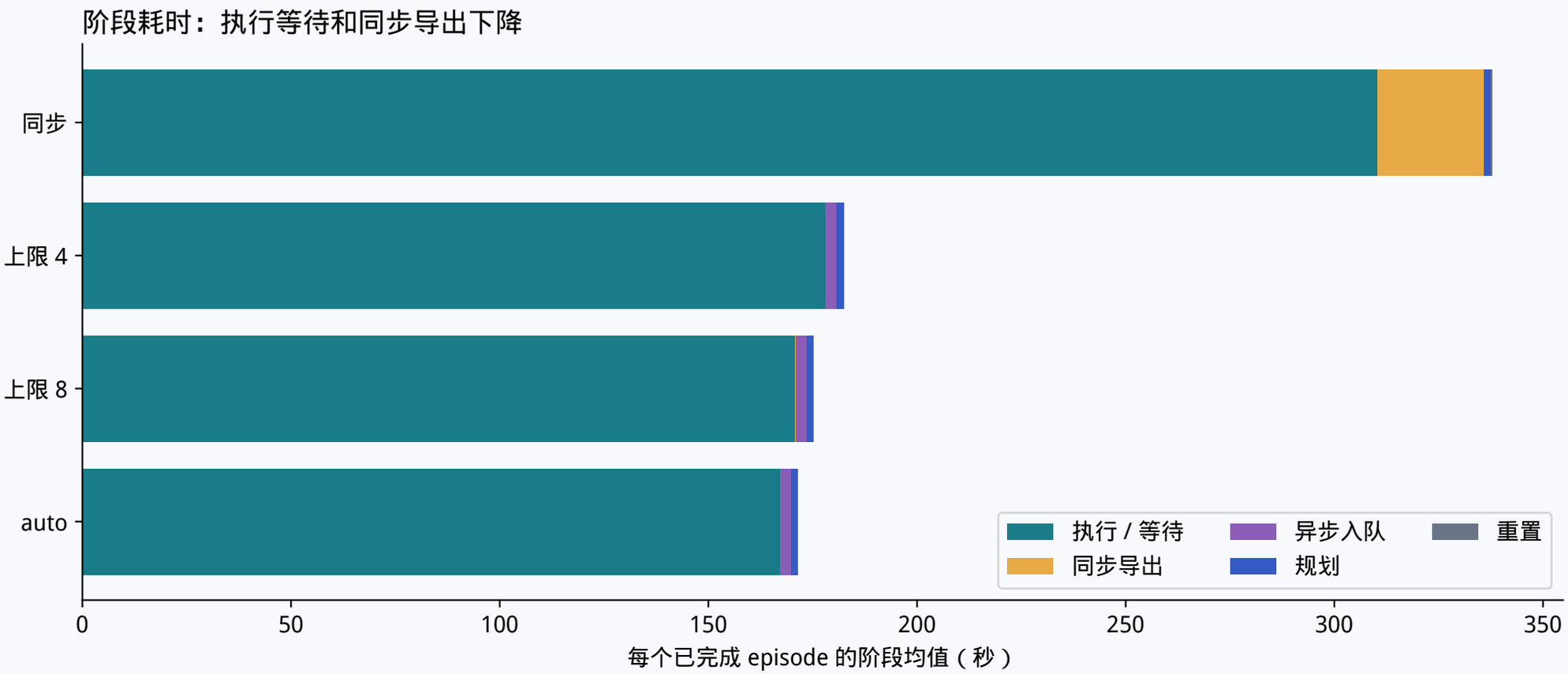
进程 A：仿真 + 规划
进程 B：仿真 + 规划
↓
共享 GPU 或绑定不同 GPU
↓
协调任务与输出

需额外设备资源与调度
进程数不等于加速倍数

设备利用率不能证明 kernel 重叠；当前没有单 GPU 多进程共享或多 GPU 部署的吞吐对照。

更多保存进程与更多 GPU 生成进程

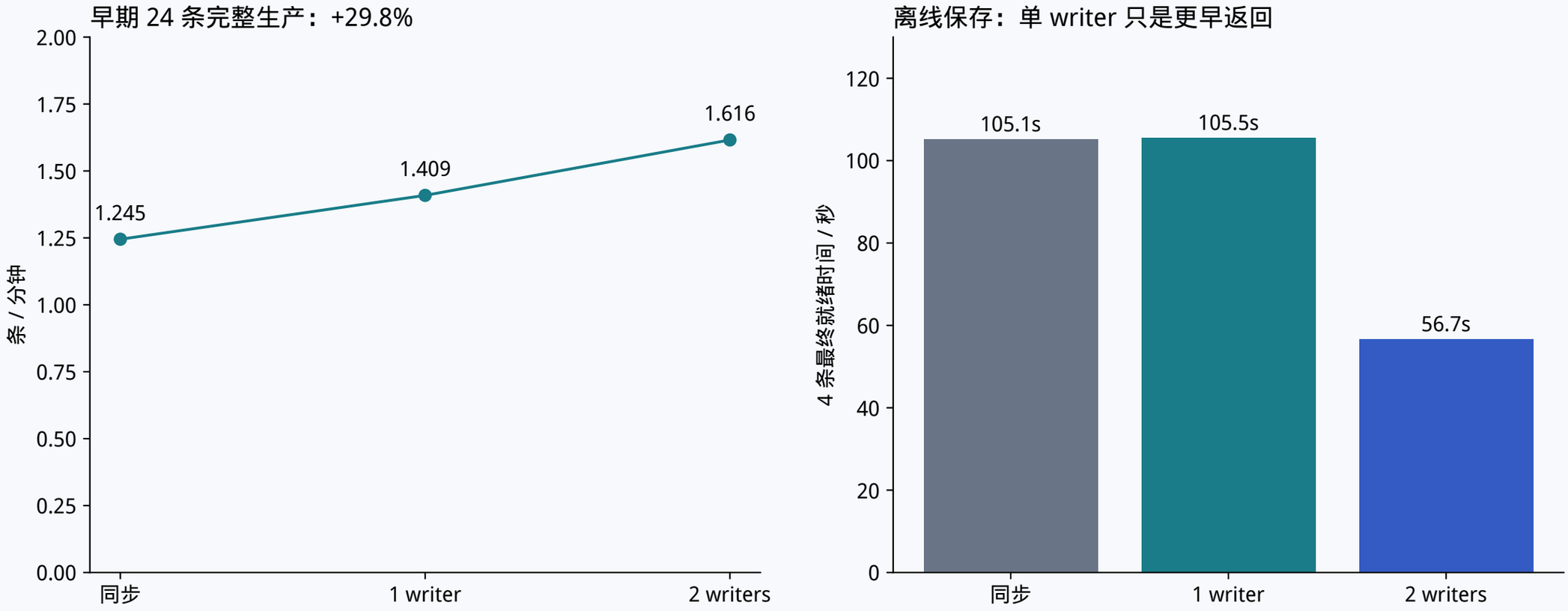
怎么做的	将当前部署与候选部署分列：当前为一个 GPU 生成主进程管理多环境，搭配 CPU writer 池；候选包括同卡多生成进程或多 GPU 分摊任务，但后两者未在本实验运行。
这样做的意义	扩展保存并行与扩展生成并行的资源约束不同。同卡增加生成进程还会争用显存和计算，多 GPU 则需要重新规划资产、任务分配和输出管理，不能沿用当前 writer 结果推算。
当前结论	目前可确认的是单生成主进程加 CPU 保存池的收益。是否适合单卡多进程或多卡部署，需要独立的显存、吞吐、质量和稳定性实验；未测方案在本图只用于比较机制。
读图与术语	同卡多进程不等于 GPU 算力增加；多个环境的批量计算也不等于多个独立生成进程。



并发 episode 的耗时会重叠；此图不等于整批墙钟时间，也不覆盖所有未计时开销。

阶段分解应怎样阅读

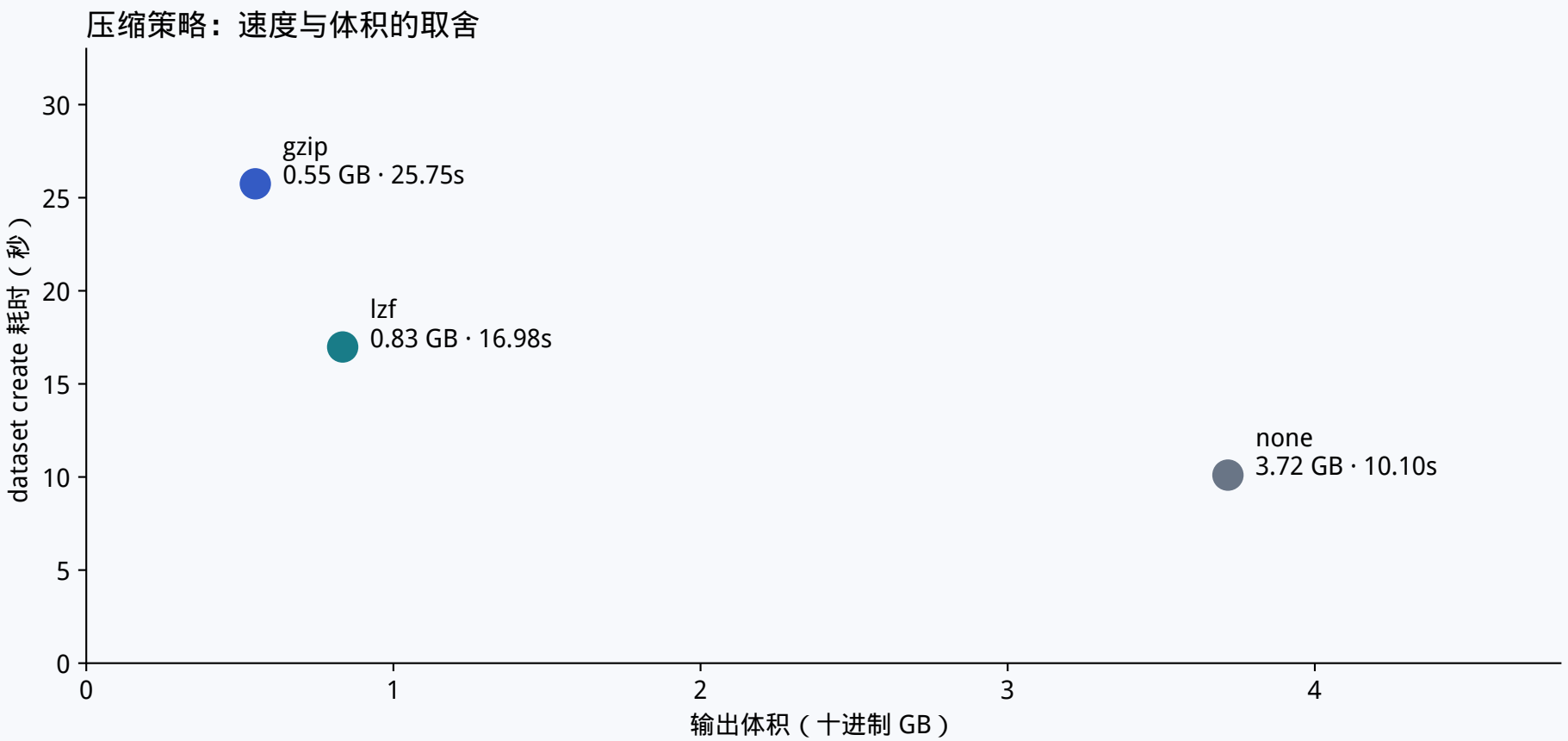
怎么做的	对历史 v1 各组 17 个完成 episode 的已记录阶段求均值，分别展示规划、执行等待、导出或入队等组成，其中包含一次生成失败。异步后台写入另列，不塞进提交阶段。
这样做的意义	阶段分解便于定位函数级变化，说明最终时间差可能从哪里产生；同时保留失败尝试，可以反映本轮实际完成任务的统计范围。
当前结论	这些均值显示同步与异步路径的阶段差异，但既不是 16 条成功数据的专属平均，也不是整批墙钟的可加分项。内部操作占比应看新增配对页，历史未记录的细节不能从本图倒推。
读图与术语	按完成 episode 计均值与按成功保存数算吞吐使用不同分母，两者回答的问题不同。



左：三个模式均为 24 条记录成功、23 条末态通过；右：CPU 离线回放，不能当作 GPU 生产吞吐。

旧生产实验和离线实验分别回答什么

怎么做的	将历史 24 episode 的生产比较，与 4 episode 的离线导出测试分开列出。旧生产各组均记录成功 24 条、严格末态通过 23 条；离线测试只用已有 4 条数据测保存路径，不进行持续生成。
这样做的意义	离线实验便于控制输入、快速筛选存储方案，生产实验则检验生成与保存争用、重叠和收尾。两者有互补价值，但时间范围和工作量不同。
当前结论	旧生产实验曾观察到双 writer 最终吞吐提高约 29.8%；该数字只对应当时协议。它与当前 8 环境四配置、4 环境内部配对属于不同实验，不能合并求平均或互相替代。
读图与术语	离线保存变快不保证生产同比例变快：保存可能已被生成掩盖，也可能通过资源争用影响生成。



1342 帧 / 相机的离线 RGB 回放，保持 chunk 可比，单轮；含压缩、HDF5 和 I/O，非完整生产。

压缩策略如何权衡速度与存储

怎么做的	使用固定输入的离线图像数据，每台相机 1,342 帧 RGB，保持分块方式一致，对照不压缩、LZF 和 gzip level 2 的写入时间与文件大小。比较范围限于本批图像和保存配置。
这样做的意义	压缩用 CPU 时间换取更少磁盘空间与写入字节。存储速度、CPU 余量和数据内容会改变权衡，离线对照可帮助选出值得放入生产实验的候选配置。
当前结论	本批固定输入中，不压缩保存更快但文件更大，LZF 提供另一种速度与体积取舍。该结果不能直接推算生成吞吐；生产选择还需核对 CPU 与磁盘争用、收尾时间、读回正确性及存储预算。
读图与术语	gzip level 2 表示压缩等级；LZF 是另一种压缩算法；chunk 是 HDF5 存储分块，改变它也可能影响性能。